

Test-Time Scaling in Reasoning LLMs: Inference Regimes, Evaluation, and Reproducibility

Mohsen Hariri Weicon Chen Nahal Shahini Vikash Singh Kai Ye
 Amirhossein Samandar Debargha Ganguly Sreehari Sankar Yanyan Zhang
 Shouren Wang Jerry Peng Biyao Zhang Michael Hinczewski Vipin Chaudhary
 {mohsen.hariri, weicon, nxs814, vikash, kxy406, axs2935, debargha, sxs2284, yxz3106, sxw992, jxp1146, bxz297, mxh605, vipin}@case.edu
 Case Western Reserve University

Abstract

Large language models can solve harder reasoning problems with more inference-time compute. The term *test-time scaling*, however, covers several inference algorithms: extending deliberation along one trajectory, sampling completed candidates and aggregating them by voting or verification, and searching over partial states. These algorithms differ in statistical structure, compute requirements, and failure modes. Treating them as interchangeable under a scalar “budget,” or reporting accuracy without specifying the inference protocol, makes results difficult to compare across studies. We study test-time scaling along three axes. First, we formalize it as budgeted inference over the implicit prefix tree of an autoregressive model and distinguish single-trajectory sequential scaling, leaf-level scaling with terminal reduction, and prefix-level scaling. Second, we treat the full inference system as the evaluated object and separate end-to-end performance from candidate-bank diagnostics. We introduce an evaluation profile whose coordinates and simple functionals recover or bound common repeated-sampling metrics, and require compute accounting and uncertainty estimates that match the protocol. Third, we distinguish exact replay from distributional reproducibility and state the requirements for each. We also organize open-weight reasoning models by model-side and interface mechanisms. Our empirical study covers broad knowledge, symbolic reasoning, and competition mathematics, and we publicly release 1,403,520 sampled model attempts.¹

Scorio <https://mohsenhariri.github.io/scorio/tts>

Datasets: 🧐 Trace 🧐 Lite 🧐 Math 🧐 SuperGPQA

1 Introduction

Large language models (LLMs) often perform better on reasoning tasks when they generate intermediate steps and use more computation at inference time. Early work on chain-of-thought prompting and self-consistency showed gains on arithmetic, symbolic, and commonsense tasks from generating multi-step solutions and aggregating samples (Wei et al., 2022; Wang et al., 2023). More recent systems make deliberate reasoning a training and interface objective rather than relying on prompting alone: DeepSeek-R1 uses reinforcement learning to elicit long-form reasoning (Guo et al., 2025); Phi-4-reasoning uses curated supervised traces, while Phi-4-reasoning-plus adds outcome-based reinforcement learning (Abdin et al., 2025); Qwen3 exposes explicit “thinking” modes and reasoning-budget controls (Yang et al., 2025a); and the gpt-oss release provides open-weight reasoning models post-trained with chain-of-thought reinforcement learning (OpenAI et al., 2025). Reasoning performance therefore depends on how computation is allocated at test time as well as on pretraining and post-training.

¹For the API list, see Appendix F; for the release data, see Appendix G.



Figure 1: A selective chronology of reasoning systems from elicited chains to budgeted inference. The top band traces single-trajectory elicitation, leaf-level sampling with terminal reduction, prefix-level search, and budget-aware control (Section 2.1). The middle band groups model and interface mechanisms into verified distillation, reward optimization, reasoning controls, and parameter-space composition (Appendices D and E). The lower bands show evaluation and reproducibility requirements (Section 3) and identify the evaluated system: checkpoint, prompt, decoder, controller or reducer, verifier or judge, budget, and stopping rule. Utility, candidate-bank profile, cost, and uncertainty are reported together. Green dots mark families represented in the experimental roster; dashed boxes denote contextual milestones. Era labels indicate shifts in emphasis rather than mutually exclusive periods.

The phrase *test-time scaling* describes several distinct inference algorithms. Some methods allocate extra compute to a single trajectory by forcing longer deliberation or intervening adaptively during generation (Muennighoff et al., 2025). Others sample completed candidates and reduce them by voting, reranking, verifier-based selection, or minimum-Bayes-risk decoding (Wang et al., 2023; Cobbe et al., 2021; Freitag et al., 2022). Still others search over partial states, as in Tree-of-Thoughts, RAP, and value-guided decoding (Yao et al., 2023; Hao et al., 2023; Yu et al., 2024). These regimes differ in statistical structure, induced proposal distributions, compute requirements, and failure modes. Treating them as interchangeable under a scalar “budget” obscures the procedure being evaluated and makes cross-paper comparisons difficult to interpret (Welleck et al., 2024; Snell et al., 2025).

Evaluating these methods requires more than benchmarking a base model in isolation. Measured performance depends on the full inference protocol: the prompt template, decoder, search controller, reducer, verifier or judge, stopping rule, and numerical settings. Best-of- N and verifier-guided reranking can improve accuracy, but selection can overoptimize imperfect proxy scores as the candidate set grows (Gao et al., 2023; Huang et al., 2025; Khalaf et al., 2025). LLM judges can evaluate open-ended outputs, but prior work documents position and verbosity biases, and their reliability depends on the task and procedure in test-time scaling settings (Zheng et al., 2023; Zhou et al., 2025). Even with fixed prompts and methods, stochastic decoding and implementation details can change benchmark estimates (Miller, 2024; Blackwell et al., 2024; Ye et al., 2024; Liu et al., 2025b). Studies of reasoning under test-time scaling must define the algorithm being evaluated and match the evaluation to the deployed inference protocol. They should also report model training, inference controls, and uncertainty with enough procedural detail for independent reruns.

Comparisons of open-weight reasoning models require system-level evaluation because their training and inference mechanisms vary. Recent releases include reinforcement-learning-first reasoning models (Guo et al., 2025), data-centric and verification-focused SFT pipelines (Ye et al., 2025; Guha et al., 2026), open reproductions built from released trace corpora and training code (Hugging Face, 2025; Open-R1 Team, 2025b), checkpoints with thinking and non-thinking modes (Yang et al., 2025a), and model-fusion systems that combine reasoning experts (FuseAI, 2025a;b). This variation confounds checkpoint comparisons because two checkpoints may differ at once in their supervision signals, reasoning styles, compute budgets, and aggregation protocols. Comparisons should separate these axes and treat reasoning as a property of the full inference system rather than of model weights alone. The chronology in Figure 1 places these developments alongside the components that determine system performance: checkpoint, interface, controller or reducer, evaluator, stopping rule, and budget.

We formalize test-time scaling as a family of budgeted inference algorithms over the implicit prefix tree of an autoregressive model. For sampled inference, the formalization separates evidence computed for each trace from decisions over a completed pool. It treats causal stopping separately because stopping changes generation cost. Shared-bank comparisons isolate aggregation, whereas end-to-end evaluation measures the full inference system. We report candidate-bank quality with repeated-sampling diagnostics that capture the trade-off between discovering a correct candidate and producing correct candidates consistently. We apply the framework to open-weight reasoning models on benchmarks in broad knowledge, symbolic reasoning, and competition mathematics.

The paper makes four contributions:

- **Formalizing test-time scaling.** We model test-time scaling as budgeted inference over the implicit prefix tree and distinguish single-trajectory sequential scaling, leaf-level scaling, and prefix-level scaling.
- **Establishing evaluation principles for test-time scaling.** We treat the base model, prompting, decoder, inference-time evidence, search or aggregation rule, stopping controller, evaluator, and budget as one inference system. We distinguish shared-bank aggregation comparisons from end-to-end evaluation and post-hoc selection from causal stopping. The evaluation principles also require compute reporting on the relevant axes and uncertainty estimates matched to the protocol.
- **Providing a model-centric view of the open reasoning ecosystem.** We organize and compare representative open-weight reasoning models by model-side and interface mechanisms: verified SFT and distillation, reinforcement learning or preference optimization, inference-time control, and parameter-space fusion.
- **Releasing a large-scale reasoning-trace resource.** We release 1,403,520 sampled attempts across competition mathematics and broad graduate-level knowledge for reasoning-behavior analysis, evaluator training or calibration, and reproducibility studies.

2 Formalizing Test-Time Scaling

Test-time scaling comprises a family of *budgeted inference algorithms* built on a fixed autoregressive model. Repeated sampling with terminal reduction, adaptive single-trajectory deliberation, and search over partial reasoning states allocate additional inference-time computation in different ways (Welleck et al., 2024; Snell et al., 2025; Muennighoff et al., 2025; Wu et al., 2024). All operate over the implicit prefix tree induced by generation: compute may be spent along one trajectory, across completed leaves, or over internal prefixes before completion. These choices define three structural regimes.

2.1 Problem setup and three-regime taxonomy

Let $x \in \mathcal{X}$ be an input prompt and let $p_\theta(\cdot | x)$ be an autoregressive language model. The finite prefixes reachable from x define an implicit rooted tree $\mathcal{T}(x)$; we write $z \preceq y$ when prefix z lies on the path to completed generation y , and $\mathcal{L}(x)$ for the set of terminal leaves (Welleck et al., 2024). A local generation

policy π , such as greedy decoding, temperature sampling, or nucleus sampling, induces a proposal distribution $q_\pi(y \mid x)$ over completed leaves. This proposal may differ from the raw model distribution because decoding can truncate, renormalize, or otherwise transform token probabilities. We treat EOS, a declared length cap, and a causal stop issued by a controller as terminal actions; a stopped output that cannot be parsed remains a terminal leaf with invalid answer $a = \perp$.

A completed generation y may contain both intermediate reasoning and a task answer. We write $\text{Parse}(y) = (r, a)$, where r is the reasoning trace and a is the extracted answer. When only part of the output can be checked programmatically, we further write $\Psi(y) = (d, s)$, where d is the deterministically verifiable component and s is the remainder. Depending on the task, d may be the final answer string, an executable program, an action sequence, or any other artifact that admits task-specific checking.

Our object of interest is a family of budgeted inference algorithms $\{\mathcal{A}_B\}_{B \in \mathcal{B}}$, where $\mathcal{B} \subseteq \mathbb{R}_{\geq 0}$ is the set of allowed budgets. Each family defines an additive cost function c with a declared unit; resources that are not commensurate are reported separately. Algorithm $\mathcal{A}_B$ may adaptively interleave primitive operations o_t such as token generation, prefix expansion, verifier calls, judge calls, or terminal reduction, subject to $\sum_t c(o_t) \leq B$. Let $\mathcal{O}(x) = \mathcal{L}(x) \sqcup \mathcal{A}_{\text{ans}}(x)$ be the disjoint union of completed leaves and answer-valued outputs. The algorithm returns $\hat{o}_B(x) \in \mathcal{O}(x)$, written $\hat{y}_B(x)$ or $\hat{a}_B(x)$ according to its type, and $U_x : \mathcal{O}(x) \rightarrow \mathbb{R}$ denotes task utility. In reasoning benchmarks U_x is often exact correctness, but the same abstraction also covers graded utilities such as execution score or preference reward.

Task utility is usually observed through evaluation signals. A programmatic verifier $V_P(x, d) \in \mathcal{V}$ may return a Boolean decision, a scalar score, a diagnostic object, or a canonicalized answer representation. A learned evaluator $J_\phi(x, \omega)$ scores either a completed leaf $\omega = y$ or a partial state $\omega = z$. Unlike programmatic checks, learned evaluators are proxies whose validity depends on their supervision and test-time protocol (Cobbe et al., 2021; Lightman et al., 2024; Zheng et al., 2023; Zhou et al., 2025).

For a task distribution $\mathcal{P}$, the associated test-time scaling curve is

$$G(B) = \mathbb{E}_{x \sim \mathcal{P}, \xi} [U_x(\hat{o}_B(x; \xi))],$$

where ξ denotes the algorithm’s internal randomness. The term *scaling* refers to how performance varies with budget; it does not imply that performance must be monotone for every method.

We use the following operational taxonomy, illustrated in Figure 2.

Single-trajectory sequential scaling. At every step, there is at most one unfinished active prefix. Additional computation changes how that prefix is extended, revised, or terminated, but never branches into a competing frontier.

Leaf-level scaling. Additional compute produces a bank of completed candidates. Any interaction between candidate trajectories is deferred until after the candidates have completed, via a terminal reducer.

Prefix-level scaling. Additional compute is allocated based on scores for unfinished prefixes. Expansion, pruning, rollout allocation, or termination decisions depend on partial states before completion.

These regimes are not mutually exclusive at the system level: many practical methods are hybrids, most commonly prefix search followed by a leaf-level reducer.

2.2 Single-trajectory sequential scaling

A single-trajectory method maintains one active state z_t and remaining budget b_t . At step t , a controller chooses a meta-action

$$\nu_t \sim \pi_{\text{seq}}(\cdot \mid x, z_t, b_t),$$

then extends only the active path,

$$z_{t+1} \sim \text{Extend}(z_t; \nu_t, p_\theta), \quad b_{t+1} = b_t - c(\nu_t).$$

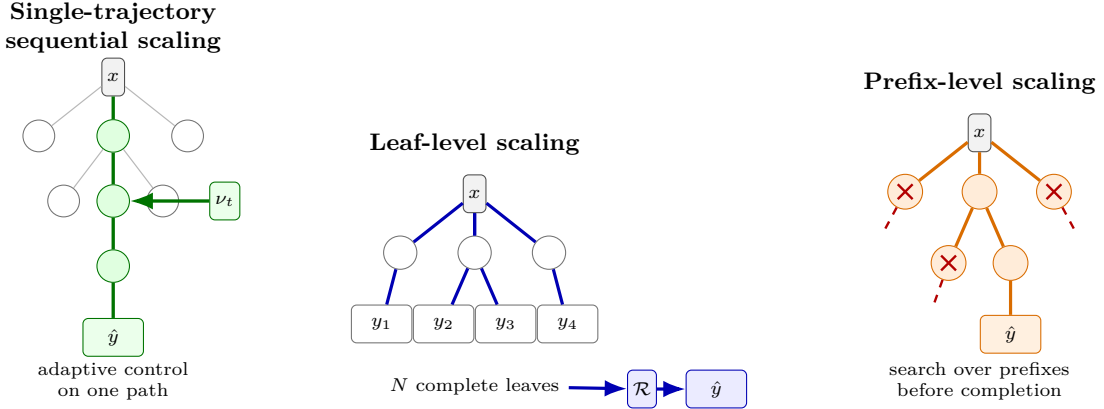


Figure 2: Three regimes of test-time scaling over the implicit prefix tree. Left: single-trajectory sequential scaling allocates compute to one evolving response without maintaining a competing frontier. Middle: leaf-level scaling allocates compute to completed root-to-leaf trajectories and reduces them to one output. Right: prefix-level scaling allocates compute to internal prefixes, expanding promising continuations and pruning others before committing to a completed leaf.

Actions include continuing generation, suppressing EOS, appending a control string or critique, changing decoding hyperparameters, or stopping. Budget-forcing methods such as s1 fit this template. Inference-time meta-generation procedures also fit when they maintain one evolving response rather than a branching frontier (Muennighoff et al., 2025; Welleck et al., 2024).

This regime has low orchestration overhead because all extra compute goes to one candidate. Without a competing unfinished prefix, however, recovery from an early misconception requires self-revision along the same path rather than a search over alternatives.

2.3 Leaf-level scaling: sampling and reduction

In leaf-level scaling, additional compute produces completed leaves and a terminal reducer chooses the output. Write $[N] = \{1, \dots, N\}$. In the canonical case,

$$\mathcal{Y}_N(x) = \{Y_i\}_{i=1}^N, \quad Y_i \stackrel{\text{i.i.d.}}{\sim} q_\pi(\cdot | x),$$

where q_π is a common proposal distribution induced by the base model and decoding policy. Appendix B extends this template to weighted and heterogeneous-proposal banks. For each candidate, let m_i contain signals available at inference time, and define

$$(r_i, a_i) = \text{Parse}(Y_i), \quad (d_i, s_i) = \Psi(Y_i), \quad v_i = V_P(x, d_i), \quad j_i = J_\phi(x, Y_i).$$

Given the proposal, the terminal stage is specified by a reducer

$$\mathcal{R}_N : (x, \{(Y_i, m_i, v_i, j_i)\}_{i=1}^N) \mapsto \hat{y} \text{ or } \hat{a}.$$

The fixed-bank leaf template separates generation from reduction. Scores assigned to one unfinished candidate cannot affect another; cross-candidate interaction begins inside $\mathcal{R}_N$ after all N leaves are complete. Adaptive stopping preserves this separation between unfinished candidates. A controller may use completed leaves to decide whether to launch another independent rollout, or it may truncate a rollout using only that rollout’s prefix-local evidence. It does not reallocate compute among competing unfinished prefixes (Section 3.3). Such controllers change the realized sample count or trace length and are not fixed-bank reducers.

2.3.1 Canonical reduction rules

Verifier-constrained selection. Let $\mathcal{I}_{\text{pass}} = \{i : V_P(x, d_i) \in \text{PASS}\}$. A verifier-constrained reducer selects

$$i^* \in \arg \max_{i \in \mathcal{I}_{\text{pass}}} T(x, Y_i, v_i, j_i),$$

where T ranks the passing candidates using, for example, the learned score j_i or sequence likelihood. The reducer uses a declared fallback if $\mathcal{I}_{\text{pass}}$ is empty. This is the generate-test-select pattern used in program-synthesis and verifier-based reasoning systems (Li et al., 2022; Cobbe et al., 2021).

For answer-valued tasks, define the eligible set $\mathcal{I}_{\text{valid}} = \{i \in [N] : a_i \neq \perp\}$. For tasks without an invalid-answer state, take $\mathcal{I}_{\text{valid}} = [N]$. Every reducer must declare a deterministic tie rule and a fallback output for $\mathcal{I}_{\text{valid}} = \emptyset$.

Answer marginalization and self-consistency. When answers or verifier outputs can be canonicalized and $\mathcal{I}_{\text{valid}} \neq \emptyset$, define the empirical answer distribution over eligible candidates

$$\hat{q}_N(a \mid x) = \frac{1}{|\mathcal{I}_{\text{valid}}|} \sum_{i \in \mathcal{I}_{\text{valid}}} \mathbf{1}[a_i = a], \quad a \neq \perp.$$

Self-consistency selects

$$\hat{a} \in \arg \max_{a \neq \perp} \hat{q}_N(a \mid x).$$

Appendix B shows that this is exactly empirical MBR with agreement utility $u(a, a') = \mathbf{1}[a = a']$ (Wang et al., 2023; Kumar & Byrne, 2004; Bertsch et al., 2023).

Learned reranking (Best-of- N). When programmatic verification is unavailable or incomplete, a standard reducer chooses among eligible candidates

$$i^* \in \arg \max_{i \in \mathcal{I}_{\text{valid}}} j_i.$$

This covers learned verifiers, reward models, and LLM judges used as inference-time rerankers (Cobbe et al., 2021; Snell et al., 2025; Huang et al., 2025; Zhou et al., 2025).

Empirical expected-utility selection. More generally, with a projection ψ and utility u , one may select

$$i^* \in \arg \max_{i \in \mathcal{I}_{\text{valid}}} \frac{1}{|\mathcal{I}_{\text{valid}}|} \sum_{k \in \mathcal{I}_{\text{valid}}} u(\psi(Y_i), \psi(Y_k)).$$

This MBR template unifies plurality vote, semantic-consensus rules, and neural-metric reranking under a common expected-utility objective (Kumar & Byrne, 2004; Freitag et al., 2022; Bertsch et al., 2023).

These rules cover the main structural forms of leaf-level reduction. Appendix B details weighted aggregation, semantic kernels, pairwise aggregation, regularized selection, and the separation of answer selection from rationale presentation.

2.3.2 Failure modes and budget allocation

Increasing the number of samples from a fixed proposal expands candidate coverage without early pruning, but the reducer introduces its own failure modes. An incomplete verifier can admit false positives as N grows, while hard argmax selection can overoptimize a misspecified learned score. This produces the non-monotone Best-of- N behavior documented in work on reward-model overoptimization and inference-time reward hacking (Gao et al., 2023; Huang et al., 2025; Khalaf et al., 2025). Judge-based reducers also inherit position and verbosity biases, so judge design and prompting are part of the method under evaluation (Zheng et al., 2023; Zhou et al., 2025). Leaf count and evaluator-call count are separate budget axes: MBR-style reducers may require $O(N^2)$ utility calls, which motivates approximations such as confidence-based pruning and approximate MBR (Cheng & Vlachos, 2023; Jinnai & Ariu, 2024).

A fixed sample count is therefore only one leaf-level design choice. Let $\hat{o}_n(x)$ denote the output of the declared size- n leaf algorithm. One may instead choose $N = N(x)$ by solving

$$N(x) \in \arg \max_{n: C_{\text{gen}}(n) + C_{\text{eval}}(n) \leq B} \mathbb{E}[U_x(\hat{o}_n(x))],$$

where C_{gen} denotes generation cost and C_{eval} covers inference-time scoring, control, and reduction. Recent studies of inference-time scaling optimize budget allocation across strategies (Wu et al., 2024); difficulty-adaptive allocation can outperform fixed- N baselines (Snell et al., 2025).

2.4 Prefix-level scaling: search over partial states

Prefix-level scaling allocates compute before trajectories are complete. To cover both token-level search and step-level methods such as Tree-of-Thoughts, RAP, and AlphaZero-like decoding, let z denote a search state representing either a literal token prefix or a macro-prefix composed of one or more reasoning steps, and write $\text{Succ}(z)$ for its allowed expansions (Yao et al., 2023; Hao et al., 2023; Wan et al., 2024b).

2.4.1 Frontier, terminal bank, and budget

A prefix-level method maintains an active frontier $\mathcal{F}_t$, a bank of completed leaves $\mathcal{Y}_t$, and remaining budget b_t . A generic iteration is

$$z_t \in \text{Select}(\mathcal{F}_t) \rightarrow \mathcal{C}_t \subseteq \text{Succ}(z_t) \rightarrow \text{score } \mathcal{C}_t \rightarrow (\mathcal{F}_{t+1}, \mathcal{Y}_{t+1}, b_{t+1}).$$

The budget may count generated tokens, node expansions, rollout calls, verifier or judge calls, or wall-clock cost. Unlike leaf-level scaling, prefix-level search uses evaluations of unfinished states to allocate compute among competing prefixes.

2.4.2 Prefix evaluation as continuation-value estimation

When a search commits to a single prefix and returns one completion drawn from it, the Bayes-optimal prefix score is the continuation value

$$Q_\rho^*(z) = \mathbb{E}_{Y \sim q_\rho(\cdot | x, z)} [U_x(Y)],$$

where $q_\rho(\cdot | x, z)$ denotes the distribution over completed leaves obtained by continuing from prefix z with rollout policy ρ . If search instead accumulates a bank that is later passed to a reducer $\mathcal{R}$, the exact marginal value of expanding z becomes history-dependent because it depends on how future leaves from z will interact with the leaves already collected. One-leaf surrogates such as correctness probability, process reward, or rollout return ignore this coupling.

2.4.3 Canonical prefix evaluators

Likelihood-based scoring. A canonical heuristic is cumulative log-probability under the local generation policy:

$$S_{\text{LL}}(z) = \sum_{j=1}^{|z|} \log \pi(z_j | x, z_{<j}).$$

Implementations sometimes modify this score with a length penalty. With $\pi = p_\theta$, the unnormalized form is the standard beam-search score and ranks prefixes by proposal probability. Length-penalized variants need not preserve that ranking. Neither form directly measures downstream utility.

Outcome-supervised value models. Outcome-supervised value models approximate the probability that a prefix reaches a correct completion and use that estimate for pruning, either directly or in combination with likelihood:

$$\hat{Q}(z) \approx \Pr_{Y \sim q_\rho(\cdot | x, z)} (U_x(Y) = 1), \quad S(z) = \hat{Q}(z) \text{ or } S(z) = \alpha S_{\text{LL}}(z) + \beta \hat{Q}(z).$$

Such value-guided decoding is explored for mathematical reasoning in OVM-style systems (Yu et al., 2024).

Process reward models (PRM). When supervision is available on intermediate reasoning steps, a process reward model can score a partial path directly. If $z = (s_1, \dots, s_t)$ is a sequence of steps, one may use

$$S_{\text{PRM}}(z) = \sum_{k=1}^t R_\phi(x, s_{\leq k}) \quad \text{or} \quad S_{\text{PRM}}(z) = R_\phi(x, z),$$

where R_ϕ scores each step in the context of the problem and the preceding steps and is learned from step-level supervision (Lightman et al., 2024).

Rollout-based evaluation. When learned value estimates are unreliable, continuation value can be estimated by Monte Carlo completion:

$$\hat{Q}_{\text{MC}}(z) = \frac{1}{M_{\text{roll}}} \sum_{m=1}^{M_{\text{roll}}} U_x(Y^{(m)}), \quad Y^{(m)} \sim q_\rho(\cdot \mid x, z).$$

MCTS-style reasoning methods can be interpreted as structured rollout allocation together with backup of these estimates (Hao et al., 2023; Wan et al., 2024b).

Sound partial checks. A partial check $H(x, z) \in \{0, 1\}$ is *sound for rejection* if

$$H(x, z) = 0 \implies U_x(y) = 0 \quad \text{for all } y \succeq z.$$

Only sound prefix tests justify safe pruning. Learned value or reward models generally do not provide this guarantee.

2.4.4 Score-induced search procedures

Beam search. Beam search keeps a frontier of size K and updates it by

$$\mathcal{F}_{t+1} = \text{TopK}\left(\bigcup_{z \in \mathcal{F}_t} \text{Succ}(z); S(\cdot)\right).$$

Standard beam search uses $S = S_{\text{LL}}$, while verifier-guided and value-guided variants replace or augment likelihood with learned prefix scores (Yu et al., 2024).

Best-first search. Best-first search repeatedly expands the highest-scoring active prefix,

$$z^* \in \arg \max_{z \in \mathcal{F}_t} S(z), \quad \mathcal{F}_{t+1} = (\mathcal{F}_t \setminus \{z^*\}) \cup \text{Succ}(z^*).$$

In classical search, admissible heuristics can yield optimality guarantees. In LLM-agent search, learned model-based value functions can be used heuristically; Koh et al. provide a best-first example (Koh et al., 2025).

MCTS-style search. Monte Carlo tree search alternates selection, expansion, evaluation, and backup. A standard UCT rule selects

$$a_t \in \arg \max_a \left(Q(z, a) + c_{\text{uct}} \sqrt{\frac{\log n(z)}{n(z, a)}} \right),$$

where $n(z)$ and $n(z, a)$ are visit counts and unvisited actions are selected first. Policy-prior variants such as PUCT replace the pure UCT bonus with one modulated by an action prior. In LLM reasoning, RAP, AlphaZero-like search, and related methods instantiate this template with rollout-based or learned value estimates (Kocsis & Szepesvári, 2006; Browne et al., 2012; Hao et al., 2023; Wan et al., 2024b). Tree-of-Thoughts can be viewed as a related search controller operating over thought-level rather than token-level expansions (Yao et al., 2023).

Additional controller variants, including learned search policies and reward-guided objectives, are discussed in Appendix C.

2.4.5 Scaling behavior and comparison to leaf-level methods

Prefix-level search can use compute more efficiently than unguided leaf sampling when prefix scores concentrate budget on high-value branches. Its risk is that pruning changes the support of reachable completions. If every ancestor of a correct leaf is pruned, no later computation can recover that leaf. For pruning, the within-instance ordering of competing frontier states often matters more than global calibration.

Under a fixed budget B , prefix search induces an effective proposal distribution over completed leaves,

$$q_{\text{search},B}(y \mid x) = \Pr(\hat{y}_B(x) = y),$$

which depends jointly on the scorer, controller, and stopping rule. When search returns an ordered bank or multiset $\mathcal{Y}$ rather than one leaf, its corresponding object is the joint law $Q_{\text{search},B}(\mathcal{Y} \mid x)$; per-leaf inclusion probabilities are only marginals and do not determine bank size, multiplicity, order, or dependence. Candidate banks returned by prefix search are therefore not i.i.d. draws from $q_\pi(\cdot \mid x)$, so direct comparisons to leaf-level Best-of- N must account for generation, evaluation, control, and decision costs.

Prefix search creates branching structures with shared prefixes. Tree-structured inference kernels can reuse key-value caches across these prefixes and thereby change the compute frontier; DEFT is one example (Yao et al., 2025a). A hybrid system can first construct a search-induced bank of completed leaves and then apply a leaf-level reducer. Such banks can be summarized by the repeated-sampling metrics in Section 3.4.

3 Evaluating Test-Time Scaling

Under test-time scaling, reported performance depends on the *full inference protocol* defined over the prefix tree in Section 2.1. The protocol determines both the *candidate distribution* and the final output rule. Its specification includes budget allocation, decoding or search, inference-time evidence, aggregation and stopping rules, and any verifier or judge calls. A report should state the benchmark structure, the induced *proposal distribution*, the aggregation and stopping protocol, the reported *estimand*, and the uncertainty procedure.

3.1 Benchmark design as evaluation structure

Benchmark design determines which capability is measured and whether additional inference-time compute can change outcomes. Under test-time scaling, benchmark suites should be organized by *evaluation structure* as well as task domain.

One distinction is between *verifiable* and *open-ended* tasks. In verifiable settings such as mathematics and code generation, correctness can be checked exactly or by execution, so repeated sampling and terminal reduction target a directly checkable criterion (Chen et al., 2021; Li et al., 2022). In open-ended settings, evaluation relies on rubrics, pairwise preferences, or judge models. Prompts, comparison order, side randomization, and aggregation can change the scores and must be reported with the results (Liang et al., 2023; Zheng et al., 2023; Chiang et al., 2024). Additional compute also affects reducers differently: verifier-based selection, agreement-based aggregation, and judge-based reranking need not improve at the same rate as the budget increases (Cobbe et al., 2021; Lightman et al., 2024; Zheng et al., 2023).

The benchmark structure should also match the task’s interaction model. Single-turn exact-answer tasks probe discovery of a correct terminal leaf, whereas interactive agent benchmarks evaluate multi-step user/tool interactions and their terminal state (Yao et al., 2025b). Inference-time algorithms may also use environment feedback, backtracking, or iterative refinement during a trajectory (Welleck et al., 2024). The same inference algorithm may score well under “any correct sample” reporting but poorly under trajectory-level reliability metrics (Section 3.4).

Benchmark difficulty should leave room for a scaling curve: if tasks are too easy, additional compute has little effect; if they are uniformly difficult, correct solutions are rarely reached and the curve remains flat (Snell et al., 2025). Because many reasoning benchmarks are small, dataset size limits the statistical resolution of scaling comparisons (Miller, 2024; Ye et al., 2024) (see Section 3.5).

3.2 Decoding protocols and induced proposal distributions

Under test-time scaling, the inference system samples from the proposal distribution induced by a complete decoding protocol rather than directly from the raw model distribution p_θ . Let Σ denote the token vocabulary. At a partial generation $h_t = (x, y_{<t})$, the model produces logits $\ell_t \in \mathbb{R}^{|\Sigma|}$ and token probabilities

$$p_t(i) = \frac{\exp \ell_t(i)}{\sum_{j \in \Sigma} \exp \ell_t(j)}, \quad i \in \Sigma.$$

A one-token decoder with configuration λ and finite state σ_t returns

$$(\tilde{q}_t, a_t^{\text{tok}}, \sigma_{t+1}) = D_\lambda(h_t, \ell_t, p_t, \chi_t, \sigma_t),$$

where χ_t denotes side information available to the decoder, such as hidden states, token embeddings, auxiliary scores, rollout estimates, or controller state. If $a_t^{\text{tok}} \neq \perp$, the next token is set deterministically to $y_t = a_t^{\text{tok}}$; otherwise $y_t \sim \text{Cat}(\tilde{q}_t)$.

A broad class of stochastic decoders admits an energy-gate form (Figure 3). For a nonnegative token measure $u_t : \Sigma \rightarrow \mathbb{R}_{\geq 0}$ and support $S_t \subseteq \Sigma$ with $\sum_{j \in S_t} u_t(j) > 0$,

$$\tilde{q}_t(i) = \text{Norm}_{S_t}(u_t)_i = \frac{u_t(i) \mathbf{1}\{i \in S_t\}}{\sum_{j \in S_t} u_t(j)}. \quad (1)$$

The measure u_t determines relative weights, while the support S_t determines which tokens remain eligible. Ordinary temperature sampling uses $u_t(i) = \exp(\ell_t(i)/T)$ and $S_t = \Sigma$; ordinary truncation rules restrict S_t and renormalize the same measure; uniform or coverage-based samplers change the weights inside the retained set. Thus support equivalence and distribution equivalence are distinct: two decoders can retain the same candidate set while inducing different token probabilities. If a gate retains zero measure, the decoder must invoke a declared fallback before applying Norm; otherwise its token kernel is undefined.

We treat EOS or STOP as an absorbing action and require termination almost surely; a hard length cap or causal truncation maps the current prefix to an explicit terminal outcome. Under these conditions, composing the local kernels gives the leaf-level proposal distribution used in Section 2.3:

$$q_\lambda(y \mid x) = \prod_{t=1}^{\tau(y)} K_{\lambda,t}(y_t \mid x, y_{<t}, \sigma_t), \quad (2)$$

where $\tau(y)$ denotes the termination step of y (its length in tokens) and $K_{\lambda,t}$ is a point mass when the decoder takes a deterministic token action and otherwise equals $\tilde{q}_t$. When the protocol π fixes a single configuration λ , we write $q_\pi = q_\lambda$; if the protocol randomizes over configurations, q_π averages q_λ over that outer randomness; if a bank uses heterogeneous protocols, then candidate i is drawn from its own proposal $q_{\pi_i}(\cdot \mid x)$ rather than from a common i.i.d. proposal.

Support construction captures the common part of many truncation rules. Each gate below takes the live candidate set A and a reference token score r , typically p_t or a monotone transform of it; under sequential composition, the renormalized $r^{(j)}$ in Equation (8) plays this role. We use three gate families. A *level gate* keeps tokens above a data-dependent threshold,

$$G_{\text{lev}}(A; r, g, \tau_g) = \{i \in A : g(i; r, \ell_t, \chi_t) \geq \tau_g(r, \ell_t, \chi_t)\}. \quad (3)$$

This family includes absolute-probability, mode-relative, entropy-scaled, and logit-band thresholds. A *head-budget gate* sorts the live set A in increasing order of a cost key κ (for probability-ranked truncation, $\kappa(i) = -r(i)$) and retains the first m_β tokens permitted by the gate parameter β (a fixed rank for top- k ; the smallest count whose retained mass reaches the target for cumulative-mass rules),

$$G_{\text{head}}(A; r, \kappa, \beta) = \{i_{(1)}, \dots, i_{(m_\beta)}\}, \quad \kappa(i_{(1)}) \leq \dots \leq \kappa(i_{(|A|)}). \quad (4)$$

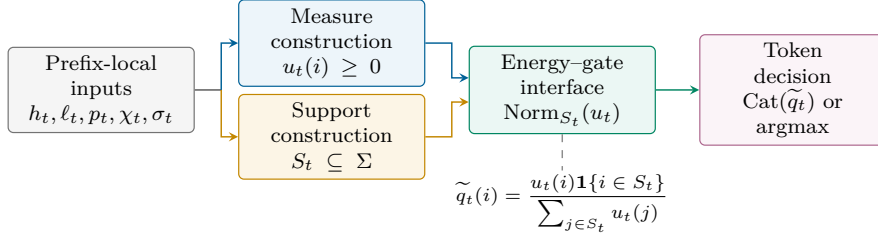


Figure 3: Elementwise view of a one-token sampling protocol. The sequence-level proposal distribution $q_\pi(\cdot | x)$ is induced by composing these local kernels until termination.

This family includes fixed-rank, cumulative-mass, curvature, entropy-budget, and typicality-based truncation. An *objective gate* chooses a support by solving, or approximating, a set objective,

$$G_{\text{obj}}(A; r, J) \in \arg \min_{\emptyset \neq A' \subseteq A} J(A'; r, \chi_t). \quad (5)$$

This separates the support decision from the weighting rule in Equation (1).

A special case is the family of mode-preserving head gates. Fix a reference distribution r_t whose rank order is the model rank order, for example p_t or a positive-temperature rescaling, and write

$$r_t(i_1) \geq r_t(i_2) \geq \dots \geq r_t(i_{|\Sigma|}), \quad K_m = \{i_1, \dots, i_m\}.$$

If each gate g in a parallel composition $\mathcal{H}$ returns a model-ranked prefix K_{m_g} , then the composed support collapses to the strictest retained prefix:

$$\bigcap_{g \in \mathcal{H}} K_{m_g} = K_{m_\cap}, \quad m_\cap = \min_{g \in \mathcal{H}} m_g. \quad (6)$$

Thus, any parallel combination of top- k (Fan et al., 2018), nucleus (Holtzman et al., 2020), probability-threshold (Hewitt et al., 2022), mode-relative, or logit-band head gates has the common form

$$\tilde{q}_t^{\mathcal{H}}(i) = \frac{u_t(i) \mathbf{1}\{i \in K_{m_\cap}\}}{\sum_{j=1}^{m_\cap} u_t(i_j)}. \quad (7)$$

The different named rules specify different ways of computing m_g ; once their head supports are evaluated in parallel, only the smallest retained prefix remains active.

This collapse does not apply to all decoders. Locally typical sampling can skip high-probability tokens because it orders tokens by the distance of their surprisal $-\log p_t(i)$ from the conditional entropy of p_t rather than by model probability (Meister et al., 2023); geometry- or objective-based crops can select non-prefix supports; and measure-changing rules can reorder candidates even when the support is fixed. Ordered composition also differs from parallel masking. For probability-only gates, a sequential protocol has the form

$$A_0 = \Sigma, \quad r^{(0)} = p_t, \quad A_j = G_j(A_{j-1}; r^{(j-1)}, \ell_t, \chi_t), \quad r^{(j)} = \text{Norm}_{A_j}(w^{(j)}), \quad (8)$$

where $w^{(j)}$ is the measure exposed by the j -th operator. Thus top- p followed by top- k need not match top- k followed by top- p , because cumulative masses, entropies, thresholds, and ranks may be recomputed after renormalization. Temperature before a mass gate can change the support, whereas temperature after a fixed support only changes within-support weights. Each sequential gate must leave positive exposed mass, or apply its declared fallback, before the next normalization.

For evaluation, these distinctions determine the estimand. In leaf-level scaling with a fixed stochastic protocol, the usual candidate bank is

$$Y_1, \dots, Y_N \stackrel{\text{i.i.d.}}{\sim} q_\pi(\cdot | x).$$

A deterministic configuration sweep instead returns a reproducible set of design points, not Monte Carlo samples from a single proposal. Prefix-level search induces its own budget-dependent proposal $q_{\text{search},B}(\cdot \mid x)$ through the scorer, controller, branching rule, rollout policy, and stopping criterion, and its returned leaves are generally dependent. Accordingly, the repeated-sampling diagnostics of Section 3.4 have an i.i.d. interpretation only when the bank is generated by independent draws from a declared fixed proposal; otherwise they are descriptive summaries of the returned bank.

A sampling report should specify the token measure, support gates, reference score space, gate ordering, deterministic actions or stochastic draws, state updates, stopping and length rules, number of samples, seeds or randomized configuration policy, and any approximation or fallback behavior. These details define the proposal distribution over reasoning traces and the test-time scaling system being evaluated.

3.3 Aggregation and stopping under sampled inference

A sampled candidate bank does not by itself define the submitted prediction. The inference protocol must say how evidence is extracted from each trace and how a fixed, completed bank is converted into an output. If either the number or length of rollouts is adaptive, the protocol must also specify when generation stops. Changing the evidence map or decision rule can improve accuracy without reducing generation cost; Figure 4 separates this completed-bank pathway from causal controllers that can avoid later rollouts or truncate the current one.

Evidence and fixed-bank decisions. For completed candidate i , let m_i contain signals available at inference time; the benchmark utility U_x is excluded. Examples include chosen-token log-probabilities, next-token distributions, process-reward scores, and verifier or judge outputs. An evidence map produces an evidence object

$$e_i = E_\eta(x, Y_i, m_i) \in \mathcal{E},$$

where $\mathcal{E}$ may be scalar or structured and E_η includes any reduction over tokens or reasoning steps required by the downstream rule. We use scalar $e_i \in \mathbb{R}$ for the score-based rules below. Sequence log-likelihood and mean token log-likelihood (Wang et al., 2023), statistics of next-token distributions (Kang et al., 2025; Fu et al., 2026), reductions of per-step process scores (Lightman et al., 2024), and external verifier or reward-model scores (Cobbe et al., 2021) are distinct evidence maps even when passed to the same selector. Pairwise methods may instead construct

$$[\mathbf{W}_n]_{ij} = u(\psi(Y_i), \psi(Y_j)).$$

A fixed-bank decision rule then returns a leaf, an answer, or a set of outputs,

$$\hat{o}_n = \mathcal{G}_\delta(x, ((Y_i, a_i, e_i))_{i=1}^n, \mathbf{W}_n).$$

Together, E_η and $\mathcal{G}_\delta$ refine the generic leaf-level reducer $\mathcal{R}_N$ of Section 2.3 into evidence and decision stages. The evidence source and decision rule are separate choices. Scores from one source can be used for reranking, filtering, or vote weighting. Conversely, a voting rule may take model likelihoods as evidence or replace them with self-evaluations or an external verifier.

For answer-valued tasks, write $a_i = \perp$ when parsing or canonicalization declares a candidate invalid, let $\mathcal{I}_{\text{valid}} = \{i : a_i \neq \perp\}$, and let $I_a = \{i \in \mathcal{I}_{\text{valid}} : a_i = a\}$. Plurality self-consistency² scores a group by $|I_a|$ (Wang et al., 2023). Hard Best-of- N chooses the largest- e_i candidate in $\mathcal{I}_{\text{valid}}$, equivalently scoring each valid answer group by $\max_{i \in I_a} e_i$ (Cobbe et al., 2021). Score-aware consensus pools evidence within a valid group using raw sums or means, softmax-normalized weights, rank weights, or calibrated log-odds (Taubenfeld et al., 2025; Kang et al., 2025; Kuang et al., 2026). Every rule must declare deterministic tie handling and a fallback submitted output when $\mathcal{I}_{\text{valid}} = \emptyset$.

Other completed-bank reducers combine filtering, resampling, and gated selection. Offline DeepConf removes low-confidence traces; Majority-of-the-Bests bootstraps Best-of- N decisions and returns their mode;

²We use *plurality voting* because this reducer selects the most frequent extracted answer even when it receives at most half of the votes. A strict majority requires more than half, so *majority voting* can be ambiguous for this rule.

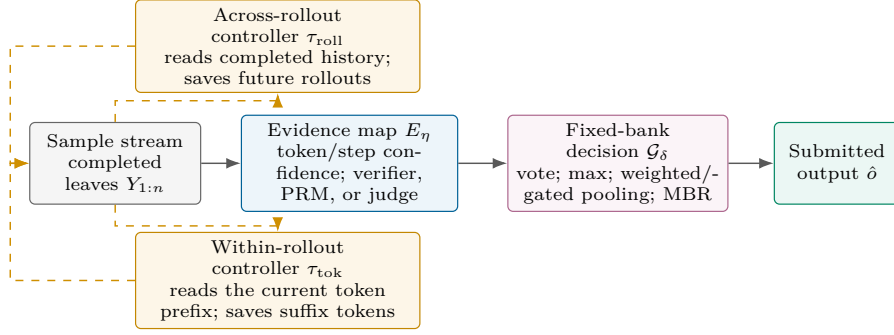


Figure 4: Aggregation and stopping under sampled inference. Evidence extraction and fixed-bank selection operate on completed traces, so they cannot save generation already spent. Causal controllers inspect only revealed history: across-rollout stopping avoids later traces, whereas token-prefix stopping can also truncate the current trace. The evidence map, controller, and final decision are all part of the evaluated protocol.

and Best-of-Majority filters answer groups by frequency before reward-ranking the retained responses for Pass@ k (Fu et al., 2026; Rakhsha et al., 2025; Di et al., 2026). When valid candidates exist, MBR selects

$$i^* \in \arg \max_{i \in \mathcal{I}_{\text{valid}}} \frac{1}{|\mathcal{I}_{\text{valid}}|} \sum_{j \in \mathcal{I}_{\text{valid}}} [\mathbf{w}_n]_{ij},$$

with the same all-invalid fallback (Kumar & Byrne, 2004; Freitag et al., 2022; Bertsch et al., 2023). Each method specifies a decision structure while leaving the evidence source separate. If the output contains both a canonical answer and a rationale, the protocol must also specify which trace accompanies the winning answer.

Score semantics are part of the algorithm. Naming the scorer does not fully specify a score-aware reducer. The protocol must state where scores come from, whether larger values are better, and how token- or step-level scores are reduced. Answer canonicalization and invalid-output handling determine which candidates enter the rule. Score transformations, temperatures, and filtering thresholds determine how evidence enters the decision. The protocol must also define ties and fallbacks, and identify any evaluator prompts or models. Calibration parameters and other tunable values must be fixed on held-out data, not test labels.

Order-only rules such as hard Best-of- N , rank-based filtering, rank voting, and Majority-of-the-Bests are invariant to strictly increasing score transformations that preserve ties. Rules based on raw sums or means, softmax weights, or log-odds are not. Sequence and mean log-probability impose different length preferences. Cross-question calibration need not predict a confidence score’s usefulness for within-question aggregation (Taubenfeld et al., 2025); calibrated log-odds may assign negative evidence to low-scoring candidates (Kuang et al., 2026). With hard maximization, exposure to proxy-score errors increases as the bank grows (Gao et al., 2023; Huang et al., 2025).

Causal stopping. Generation cost can fall only when a causal controller stops future work; evidence extraction and fixed-bank selection act after their required traces are complete. Let

$$\mathcal{H}_n = \sigma(x, (Y_i, a_i, m_i, e_i)_{i \leq n}, \mathbf{W}_n, \zeta_{0:n})$$

be the information available after n completed rollouts, including all revealed evidence, pairwise values, and controller random state $\zeta_{0:n}$. An across-rollout rule is a bounded stopping time $\tau_{\text{roll}} \leq N_{\text{max}}$ satisfying $\{\tau_{\text{roll}} \leq n\} \in \mathcal{H}_n$, and its final decision may use only the bank observed through τ_{roll} . In plain terms, whether the procedure has stopped by rollout n may depend on the first n completed rollouts, but not on any later candidate or inference-time signal. Adaptive-Consistency stops when the posterior probability that the current count leader has greater latent mass than its runner-up crosses a threshold, whereas Early-Stopping Self-Consistency stops after a valid unanimous answer window (Aggarwal et al., 2023; Li et al.,

2024). The sample cap $N_{\max}$ is separate from the cumulative budget B : before launching or continuing an operation, the controller must establish that its declared worst-case charge keeps total cost at most B . We assume operations that would overshoot are not launched; a protocol using causal truncation instead must declare that terminal rule. Warm-up, discarded, evaluator, controller, and final-decision work all consume the same declared budget unit.

For rollout i , let $\mathcal{F}_{i,t}$ contain the permitted prior history, revealed evaluator/controller state, and the current rollout’s information through token t . A within-rollout stopping time τ_{tok} satisfies $\{\tau_{\text{tok}} \leq t\} \in \mathcal{F}_{i,t}$: it cannot use the hidden suffix or eventual answer to decide whether to truncate the current trace. Truncation emits an explicit terminal outcome, which may parse as invalid, rather than treating an unfinished prefix as a completed leaf. DeepConf, for example, uses completed warm-up traces to calibrate a confidence threshold and may terminate later traces when sliding-window token confidence falls below it (Fu et al., 2026). Across-rollout rules can avoid only later traces, whereas within-rollout rules can also avoid a suffix of the current trace. Offline filtering of completed traces may change the returned answer but saves no generation tokens.

Comparison and compute accounting. A *shared-bank* comparison measures aggregation conditional on one completed bank. Each offline protocol receives the same candidates and only the inference-time signals it is permitted to use. Holding E_η , parsing, canonicalization, candidate eligibility, and score transformations fixed isolates $\mathcal{G}_\delta$; otherwise, those differences belong to the compared post-generation protocols. If both evidence and decision components vary, the comparison covers the full post-generation stage ($E_\eta, \mathcal{G}_\delta$) under a fixed proposal and realized bank. Neither variant can estimate savings from adaptive stopping or changes caused by a different generation policy. Generalization beyond the realized bank still requires uncertainty estimates over prompts and candidate draws. An *end-to-end* comparison instead allows each complete protocol to generate its own stream, acquire its own evidence, execute its stopping rule, and be evaluated by M_B .

For one declared additive work unit, the matched budget decomposes as

$$C_{\text{total}} = C_{\text{gen}} + C_{\text{eval}}, \quad C_{\text{eval}} = C_{\text{signal}} + C_{\text{control}} + C_{\text{decision}}.$$

This accounting includes warm-up and discarded tokens, verifier or judge calls, repeated controller evaluations, and aggregation itself. Equal sample counts need not imply equal compute: external scoring adds candidate-wise cost, and naive pairwise MBR requires $O(n^2)$ utility evaluations (Cheng & Vlachos, 2023; Jinnai & Ariu, 2024). Latency under parallel or overlapping stages, throughput, peak memory, and other noncommensurate resources are not additive terms in this equation and should be reported separately alongside any scalar work budget. End-to-end utility should be plotted against total cost. Shared-bank results instead identify differences attributable to the post-generation aggregation stage, and holding evidence fixed narrows attribution to the decision rule. The discovery–stability profile in Section 3.4 remains complementary because it describes candidate availability under the proposal, not the success probability of a particular aggregation protocol.

3.4 Evaluation targets and metrics

Evaluation under test-time scaling should separate the performance of the deployed inference system from the shape of the candidate bank it induces.

End-to-end system performance. Let $\mathcal{P}$ denote the task distribution and $\mathcal{D}_{\text{eval}} = \{x_q\}_{q=1}^Q$ the evaluation set. For task utility U_x , budgeted algorithm $\mathcal{A}_B$, and algorithmic randomness ξ ,

$$M_B = \mathbb{E}_{x \sim \mathcal{P}, \xi} [U_x(\hat{o}_B(x; \xi))], \quad \widehat{M}_B = \frac{1}{Q} \sum_{q=1}^Q U_{x_q}(\hat{o}_B(x_q; \xi_q)).$$

This is the estimand for capability claims: it is the scaling curve $G(B)$ of Section 2.1, with the algorithmic randomness ξ written explicitly. Pairwise win rates, judge-aggregated rankings, and other relative comparisons define separate estimands; when they are used, the judge prompt, response order, reference material, and aggregation rule are part of the evaluation protocol.

Discovery–stability profile. For repeated-sampling diagnostics, fix the proposal or inference protocol that produces completed candidates. Let $Z_{q,i} \in \{0, 1\}$ indicate whether candidate i for prompt q is correct. Under the independent-attempt abstraction, let $p_q = \Pr(Z_{q,i} = 1 \mid x_q)$ be the latent single-attempt success probability for prompt q . For k fresh attempts,

$$X_{q,k} \mid p_q \sim \text{Binomial}(k, p_q).$$

For threshold $t \in \{1, \dots, k\}$, define the binomial tail kernel

$$\kappa_{k,t}(p) = \Pr\{\text{Binomial}(k, p) \geq t\} = \sum_{j=t}^k \binom{k}{j} p^j (1-p)^{k-j}.$$

The dataset-level discovery–stability profile at budget k is

$$S_k = (S_{k,1}, \dots, S_{k,k}), \quad S_{k,t} = \frac{1}{Q} \sum_{q=1}^Q \kappa_{k,t}(p_q).$$

Thus $S_{k,t}$ is the expected fraction of prompts on which a fresh k -attempt evaluation produces at least t correct candidates. Low thresholds measure discovery, high thresholds measure repeatability, and the full profile records how quickly occasional success decays into stable success.

When an observed bank contains N candidates per prompt and $k \leq N$, let $\nu_q = \sum_{i=1}^N Z_{q,i}$. The corresponding finite-bank diagnostic is the without-replacement tail

$$\hat{S}_{k,t}^{\text{bank}} = \frac{1}{Q} \sum_{q=1}^Q \sum_{j=t}^k \frac{\binom{\nu_q}{j} \binom{N-\nu_q}{k-j}}{\binom{N}{k}},$$

with invalid binomial coefficients interpreted as zero. This statistic evaluates the threshold event for a size- k subset drawn without replacement from the observed bank, whereas S_k is the prospective latent profile. For banks produced by adaptive search or other dependent procedures, the finite-bank profile remains a descriptive summary of the returned bank; the latent i.i.d. interpretation should be invoked only when it matches the sampling protocol.

A Bayesian report separates finite-bank evidence from the latent profile. With $p_q \sim \text{Beta}(\alpha_q^0, \beta_q^0)$, defaulting to $\text{Beta}(1, 1)$ unless an auxiliary prior bank is declared, conjugacy gives

$$p_q \mid Z_{q,1:N} \sim \text{Beta}(a_q, b_q), \quad a_q = \alpha_q^0 + \nu_q, \quad b_q = \beta_q^0 + N - \nu_q.$$

The posterior mean of each profile coordinate is the beta-binomial predictive tail

$$\mu_{k,t} = \frac{1}{Q} \sum_{q=1}^Q \sum_{j=t}^k \binom{k}{j} \frac{B(a_q + j, b_q + k - j)}{B(a_q, b_q)}, \quad t = 1, \dots, k,$$

where $B(\cdot, \cdot)$ is the beta function. Shared posterior draws of $p_{1:Q}$ should be used to evaluate all thresholds and all scalar summaries, preserving their posterior dependence.

For rubric-valued outcomes, the same construction replaces the Bernoulli model with a Dirichlet–categorical model, fixes a category score vector in advance, and computes tail probabilities of the normalized k -sample rubric score. The binary exact-match profile above is the two-category case with score vector $(0, 1)$.

Scalar views of the profile. The profile S_k is the primary bank-level object. A scalar metric is a prespecified functional $f(S_k)$, not a replacement for the profile. For a linear threshold utility with weights $\omega_{t,k} \geq 0$ and $\sum_{t=1}^k \omega_{t,k} = 1$,

$$U_\omega(k) = \sum_{t=1}^k \omega_{t,k} S_{k,t}.$$

Common repeated-sampling metrics are coordinates or simple functionals of the same profile, or are bounded by them:

$$\text{Pass}@k = S_{k,1}, \quad \text{pass}^k = S_{k,k}, \quad \text{Maj}@k \geq S_{k, \lfloor k/2 \rfloor + 1} \quad (\text{single canonical target}),$$

$$\text{G-Pass}@k_\tau = S_{k, \lceil \tau k \rceil}, \quad 0 < \tau \leq 1,$$

$$\text{mG-Pass}@k = \frac{1}{k - \lfloor k/2 \rfloor} \sum_{t=\lfloor k/2 \rfloor + 1}^k S_{k,t}, \quad \text{Geom}@k = (S_{k,1} S_{k,k})^{1/2},$$

where the averaged form of $\text{mG-Pass}@k$ coincides with the definition of Liu et al. (2025a) for even k . The finite-bank versions are obtained by replacing $S_{k,t}$ with $\hat{S}_{k,t}^{\text{bank}}$; in particular, $\widehat{\text{Pass}}@k = \hat{S}_{k,1}^{\text{bank}}$. The strict-majority coordinate $S_{k, \lfloor k/2 \rfloor + 1}$ lower-bounds plurality voting: a strict majority of correct candidates guarantees that the correct answer wins the vote, whereas plurality can also succeed with fewer correct candidates when incorrect answers disagree. Ordinary single-sample pass rate is $S_{1,1}$, and for any k the uniform-threshold area recovers the same quantity:

$$\frac{1}{k} \sum_{t=1}^k S_{k,t} = \frac{1}{Q} \sum_{q=1}^Q p_q.$$

These quantities characterize candidate availability under the proposal; they do not give the success probability of an aggregation rule. Selection rules such as self-consistency, verifier reranking, MBR, and judge-based selection must be evaluated end-to-end through M_B . The same applies to adaptive stopping, which changes the realized bank. Under the complete protocol in Section 3.3, S_k remains a complementary diagnostic of the candidate bank and its discovery–stability shape.

3.5 Reproducibility

Under test-time scaling, a candidate bank and its submitted output together form one realization of a randomized inference protocol. *Exact replay* reconstructs the same candidates, inference-time signals, stopping and aggregation decisions, and scores under fixed inputs, random streams, software, and hardware. *Distributional reproducibility* instead asks whether an independent rerun returns a statistically compatible estimate of the same estimand, such as M_B , the profile S_k , or a judge-defined win rate. A fixed seed can support exact replay, but it does not measure variation over prompts, candidate draws, judge calls, or numerical execution (Bethard, 2022).

Candidate banks are not always independent. Shared prefix-search state, verifier feedback, tool state, or controller decisions can couple candidates; adaptive stopping can also make bank size depend on earlier outcomes. In these cases, the latent i.i.d. interpretation of S_k does not apply, although the finite-bank profile remains descriptive (Section 3.4). A replay record should include the raw candidates, inference-time signals, stopping and aggregation decisions, and a random-stream identifier for each candidate. Deriving each candidate’s random state from the run, prompt, sample index, and stream name prevents asynchronous batching from reassigning random variates across candidates.

Quantifying variability. A distributional reproducibility claim should specify a compatibility criterion and the random sources it covers. A prompt bootstrap over observed candidate banks estimates variation due to prompt composition conditional on those banks; it does not include new candidate draws, judge calls, or numerical execution (Efron, 1979). Claims that average over these sources require independent protocol reruns or a resampling design that includes the corresponding stages (Bouthillier et al., 2021; Blackwell et al., 2024). Each replicate must recompute the reported bank statistic, reducer, or stopping rule rather than resample an already aggregated score. Otherwise, uncertainty can be narrower than rerun variability, and system rankings can change across runs (Miller, 2024; Ye et al., 2024; Hariri et al., 2026b;a).

Study block	Questions	Banks	N	Responses	Other recorded signals
MMLU-Pro + BBH	18,543	27	1	500,661	Parsed answers
AIME'24/'25, HMMT'25, and BrUMO'25	120	20	80	192,000	Chosen-token log probability and rank; outcome-verifier score
AIME'26, HMMT Nov.'25/Feb.'26, CMIMC'25, and SMT'25	186	7	80	104,160	Top-20 token probabilities; reference-based and reference-free verifiers
SuperGPQA (subset)	3,600	4	80	1,152,000	Top-20 token probabilities and both verifier families
Total	22,449	–	–	1,948,821	

Table 1: **Response-bank inventory.** A bank is one model configuration evaluated over a complete question set; N is the number of responses per question in that bank. The signal-rich mathematics row includes three independently generated medium-effort repeats used only for repeated-run diagnostics.

Numerical execution. Precision, kernels, and batching can change a near-tied next-token decision and propagate the difference through a reasoning trace; even greedy decoding may therefore fail to replay exactly (Hochlehnert et al., 2025; Tahmasivand et al., 2025; Yuan et al., 2025). An exact-replay report should specify the inference library and version, numerical formats, quantization, kernels, and batch schedule. A matched FP32 or higher-precision audit using the same prompts and random-stream assignments can estimate sensitivity to numerical format, but does not establish deterministic replay (Yuan et al., 2025).

4 Empirical Study and Trace Corpus

Our empirical study uses fixed response banks to measure how quickly additional sampling reveals a correct candidate and how often practical reducers select one. The public release retains the responses, outcomes, and recorded signals for three collections, so new reducers can be evaluated without regenerating those responses. The empirical study has four blocks spanning different tasks, model rosters, sampling protocols, and levels of signal detail (Table 1). We therefore report each block separately.

4.1 Broad evaluation across knowledge and symbolic reasoning

This block covers 14 MMLU-Pro domains (Wang et al., 2024) and the 23 BIG-Bench Hard tasks (Suzgun et al., 2023). We evaluate 27 open-weight reasoning models with one zero-shot chain-of-thought response per question, yielding 500,661 responses. Across the roster, MMLU-Pro exact match ranges from 3.13% to 70.47%, and BBH flexible-extraction exact match ranges from 25.97% to 82.20%. We report the suites separately because their tasks, answer spaces, and parsers differ.

4.2 Candidate discovery outpaces answer selection

This block contains 80 seeded responses for every combination of 20 model configurations and 120 questions from AIME'24, AIME'25, HMMT'25, and BrUMO'25 (Jia, 2024; Zhang & Math-AI, 2025; MathArena, 2025c;a), for 192,000 responses. We compute exact candidate-bank statistics under without-replacement sampling at $k \in \{1, 2, 4, 8, 16, 32, 64, 80\}$, and replay reducers on nested random subsets of the same banks. Because every reducer sees subsets of the same response banks, differences among reducers cannot be attributed to newly generated candidates.

Across the roster, the median Pass@ k rises from 56.49% at $k = 1$ to 82.08% at $k = 80$, whereas the median all-correct coordinate falls to 15.00%. For this roster, mean Spearman agreement with the $N = 80$ accuracy ranking is 0.967 at $k = 1$ and 0.992 at $k = 8$. These values describe the 20 configurations and their sampled banks; they do not establish a general small-sample guarantee.

Qwen3-30B-A3B-Thinking-2507 has the highest single-response accuracy in the roster, at 75.56%. At $k = 80$, Pass@80 is 91.67% and the finite-bank all-correct coordinate is 43.33% (Figure 5, left). Literal answer

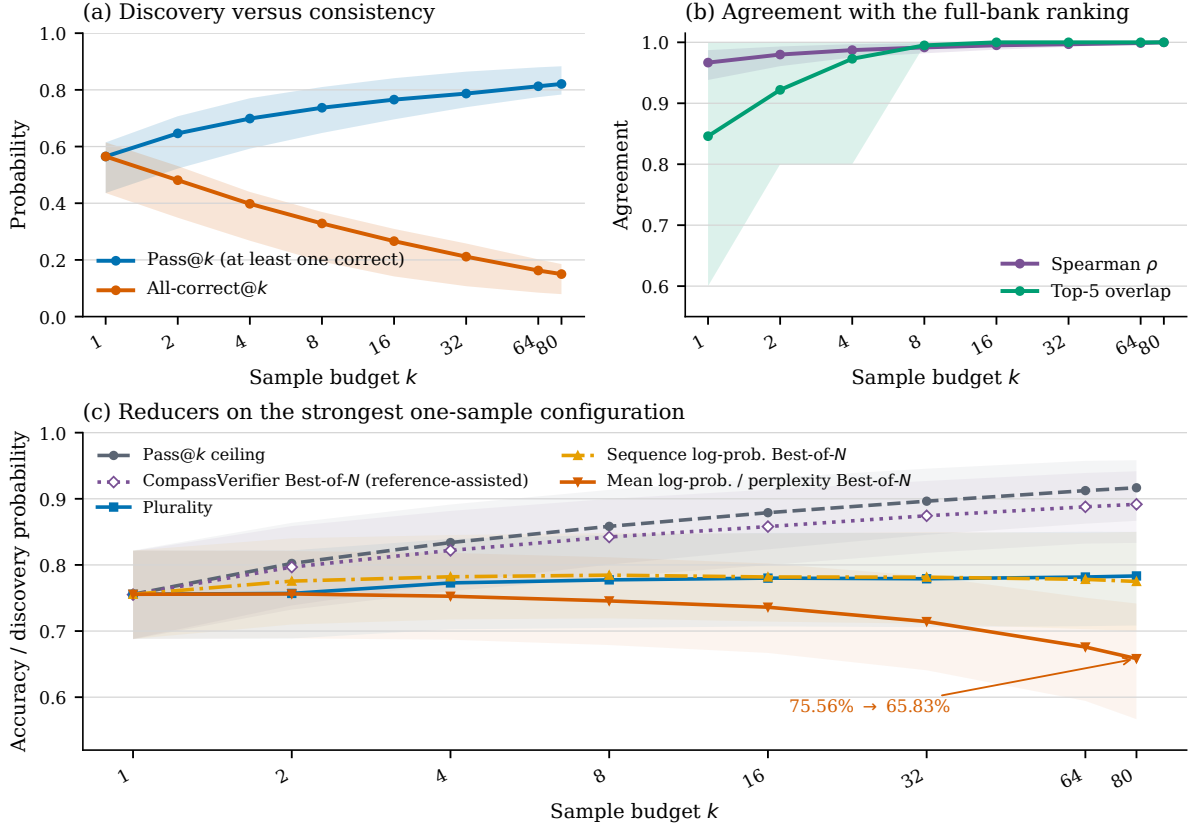


Figure 5: **Candidate availability and reducer accuracy under repeated sampling.** (a) Median exact Pass@ k and pass k across the 20 configurations; bands span the interquartile range of configurations. (b) Mean agreement of subset rankings with the $N = 80$ ranking; bands are central 95% ranges over 200 paired subset replays. (c) Reducers for Qwen3-30B-A3B-Thinking-2507, the strongest single-response configuration in this roster; bands are 95% prompt-bootstrap intervals. CompassVerifier sees the reference answer and is shown only as a reference-assisted diagnostic. Mean log probability and negative perplexity induce the same ordering, so one curve represents both.

plurality reaches 78.33%, and sequence-log-probability selection reaches 77.50%. Accuracy from selecting the response with the highest mean token log probability instead falls from 75.56% to 65.83% as the bank grows (Figure 5, right). For this score, expanding the candidate bank from 1 to 80 therefore lowers submitted-answer accuracy despite raising Pass@ k . The reference-assisted CompassVerifier diagnostic reaches 89.17%, 2.50 percentage points below Pass@80, but it has access to the gold answer and is not an inference-time result.

4.3 Full trace collection on competition mathematics

This block contains 186 problems from five competition datasets distributed by MathArena (Dekoninck et al., 2026): AIME’26, HMMT Feb.’26, HMMT Nov.’25, CMIMC’25, and SMT’25 (MathArena, 2026a;b; 2025d;b;e).

The effort comparison comprises one Qwen3.6-35B-A3B (Qwen Team, 2026) bank and three gpt-oss-20b banks at low, medium, and high reasoning effort. Every trace records the full response, rule-based outcome, chosen token, and up to 20 token alternatives at every generation position.

Pass@ k increases for all four banks (Figure 6). At $k = 80$, Pass@80 is 94.62% for Qwen3.6 and 72.58%, 91.94%, and 93.55% for gpt-oss low, medium, and high. The corresponding accuracies after reference-free

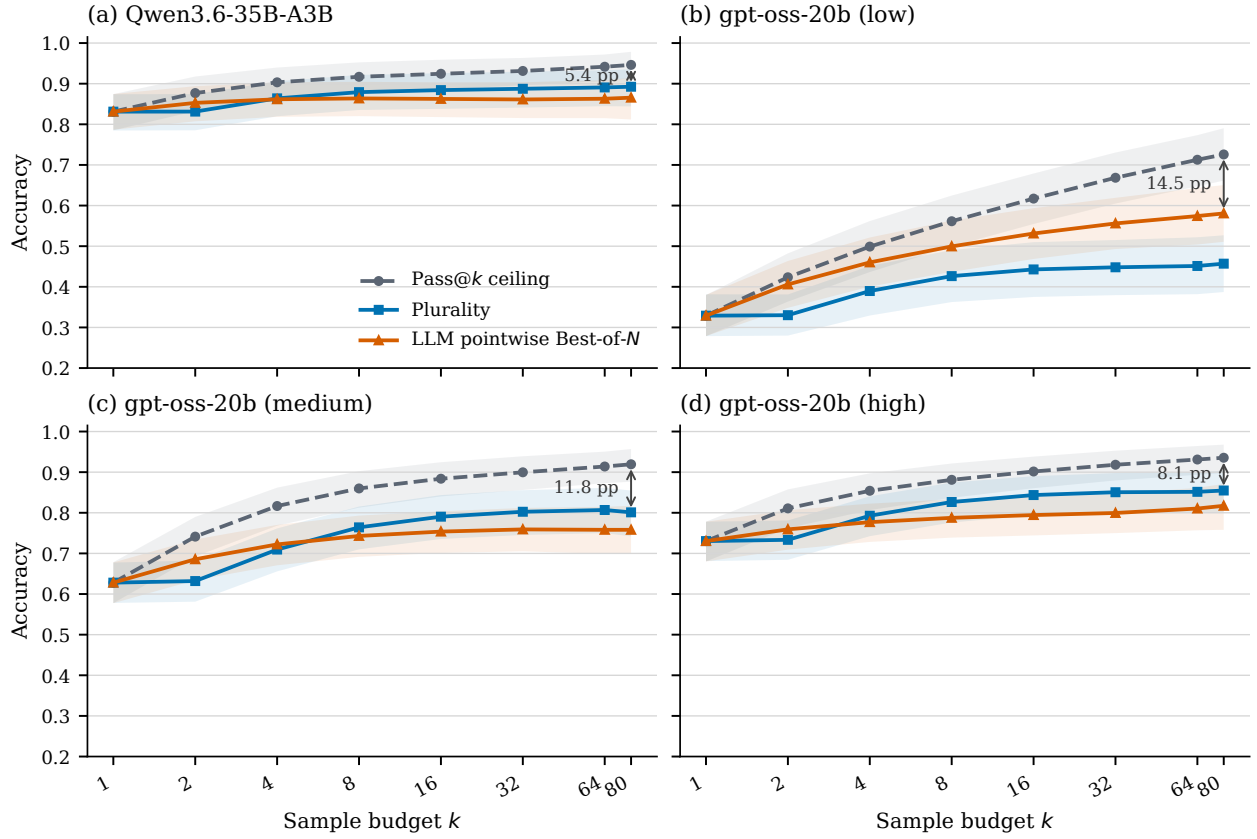


Figure 6: **Finite-bank scaling on the five 2025–2026 competition sets.** Each panel compares the exact Pass@ k ceiling with literal answer plurality and pointwise Best-of- N using the reference-free verifier described below. Arrows mark the $k = 80$ gap between Pass@80 and the better of the two observed reducers. Shading gives 95% prompt-bootstrap intervals conditional on the observed 80-response banks. Intermediate reducer points average 2,000 nested subset replays; $k = 1$ and $k = 80$ use exact endpoints. The horizontal axis counts generated candidates and excludes verifier computation.

pointwise selection are 86.56%, 58.06%, 75.81%, and 81.72%. The remaining 8.06–16.13 percentage-point gaps measure the share of questions for which a correct candidate is available but the reducer does not select it. Overlapping prompt-bootstrap intervals and differences in response length preclude a total ordering of the generators. Among high-effort responses, 2,399 of 14,880 (16.12%) reach the common 81,920-token cap, compared with none at low effort.

4.4 Reference-assisted and reference-free verifier signals

We compute one reference-assisted score and one reference-free score. CompassVerifier-3B (OpenCompass) receives the question, reference answer, and candidate response and predicts A/B/C for correct, incorrect, or invalid responses (Liu et al., 2025c). We retain both its direct label distribution and a null-baseline-adjusted contextual distribution. The reference-dependent score is an outcome diagnostic and cannot be used for inference-time selection when gold answers are unavailable.

The reference-free score is a pointwise adaptation of the pairwise LLM-as-a-Verifier method (Kwok et al., 2026). Qwen3.6 receives one problem and one candidate response, with no reference answer, ground truth, Compass result, or other candidate. It scores problem understanding, reasoning validity, and conclusion support on an ordinal A–T scale. We map A through T to 20 through 1, compute the expected score from the returned, visible-mass-renormalized scoring-token probabilities, normalize each criterion to $[0, 1]$, and average the three values. The score can rank candidates without a gold answer.

Against the rule-based outcome on 104,160 competition-mathematics responses, the trace-level ROC AUC is 0.983 for direct Compass A, 0.977 for contextual Compass A, and 0.871 for the pointwise score (Figure 7). The pointwise AUC is 0.744 on Qwen-generated responses and 0.876 on gpt-oss responses.

4.5 SuperGPQA

For SuperGPQA (M-A-P et al., 2025), we select 50 questions from each of 72 EvalScope fields, giving 3,600 questions (ModelScope Team, 2024). The four banks contain 80 responses per question from Qwen3.6 and gpt-oss at low, medium, and high effort, for $3,600 \times 4 \times 80 = 1,152,000$ responses.

For the gpt-oss high bank, mean response accuracy is 45.03%, Pass@80 is 81.94%, and pass⁸⁰ is 9.47%. Of the 3,600 questions, 650 have no correct response and 341 are answered correctly in all 80 responses. Field-level response accuracy ranges from 21.33% in Aquaculture to 76.78% in Mathematics (Figure 8).

5 Conclusion

Test-time scaling encompasses a family of budgeted inference algorithms. Viewing these algorithms as operations over the implicit prefix tree distinguishes among single-trajectory deliberation, leaf-level sampling with terminal reduction, and prefix-level search. It also identifies the proposal, evidence map, decision rule, and causal stopping controller as distinct components of the inference process. Accordingly, the complete inference system is the scientifically meaningful unit of comparison.

This perspective clarifies how reasoning systems should be evaluated. End-to-end utility and candidate-bank diagnostics answer different questions; shared-bank and end-to-end comparisons support different claims; compute accounting must include generation, evaluation, control, and decision; and uncertainty estimates must reflect the deployed inference protocol. Our framework also unifies repeated-sampling evaluation through the discovery–stability profile and distinguishes exact replay from distributional reproducibility. We analyze and publicly release 1,403,520 model attempts sampled from benchmarks covering broad knowledge, symbolic reasoning, and competition mathematics.

Our empirical study demonstrates the practical consequences of this system-level view. Model rankings change with the rubric, sampling budget, benchmark, and commitment standard; inference-time signals that predict correctness within individual trajectories need not reliably rank models; and strict answer extraction can conflate abstention with error. These results suggest that progress in test-time scaling should be assessed through reproducible inference systems with explicitly reported inference protocols, compute budgets, and uncertainties, rather than through checkpoint scores obtained under incompletely specified procedures.

Acknowledgments

This research was supported in part by NSF awards 2117439, 2112606, and 2320952.

References

- Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J. Hewett, Mojan Javaheripi, Piero Kauffmann, James R. Lee, Yin Tat Lee, Yuanzhi Li, Weishung Liu, Caio C. T. Mendes, Anh Nguyen, Eric Price, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Xin Wang, Rachel Ward, Yue Wu, Dingli Yu, Cyril Zhang, and Yi Zhang. Phi-4 technical report. Technical Report MSR-TR-2024-57, Microsoft, December 2024. URL <https://www.microsoft.com/en-us/research/publication/phi-4-technical-report/>.
- Marah Abdin, Sahaj Agarwal, Ahmed Awadallah, Vidhisha Balachandran, Harkirat Behl, Lingjiao Chen, Gustavo de Rosa, Suriya Gunasekar, Mojan Javaheripi, Neel Joshi, Piero Kauffmann, Yash Lara, Caio César Teodoro Mendes, Arindam Mitra, Besmira Nushi, Dimitris Papailiopoulos, Olli Saarikivi, Shital Shah, Vaishnavi Shrivastava, Vibhav Vineet, Yue Wu, Safoora Yousefi, and Guoqing Zheng. Phi-4-reasoning technical report. arXiv preprint arXiv:2504.21318, 2025. URL <https://arxiv.org/abs/2504.21318>.

-
- Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Let’s sample step by step: Adaptive-Consistency for efficient reasoning and coding with LLMs. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12375–12396, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.761. URL <https://aclanthology.org/2023.emnlp-main.761/>.
- Wasi Uddin Ahmad, Somshubra Majumdar, Aleksander Ficek, Sean Narenthiran, Mehrzad Samadi, Jocelyn Huang, Siddhartha Jain, Vahid Noroozi, and Boris Ginsburg. OpenCodeReasoning-II: A simple test time scaling approach via self-critique. arXiv preprint arXiv:2507.09075, 2025a. URL <https://arxiv.org/abs/2507.09075>.
- Wasi Uddin Ahmad, Sean Narenthiran, Somshubra Majumdar, Aleksander Ficek, Siddhartha Jain, Jocelyn Huang, Vahid Noroozi, and Boris Ginsburg. OpenCodeReasoning: Advancing data distillation for competitive coding. arXiv preprint arXiv:2504.01943, 2025b. URL <https://arxiv.org/abs/2504.01943>.
- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12248–12267, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.662. URL <https://aclanthology.org/2024.acl-long.662/>.
- Kyunghoon Bae, Eunbi Choi, Kibong Choi, Stanley Jungkyu Choi, Yemuk Choi, Kyubeen Han, Seokhee Hong, Junwon Hwang, Taewan Hwang, Joonwon Jang, Hyojin Jeon, Kijeong Jeon, Gerrard Jeongwon Jo, Hyunjik Jo, Jiyeon Jung, Euisoon Kim, Hyosang Kim, Jihoon Kim, Joonkee Kim, Seonghwan Kim, Soyeon Kim, Sunkyoung Kim, Yireun Kim, Yongil Kim, Youchul Kim, Edward Hwayoung Lee, Gwangho Lee, Haeju Lee, Honglak Lee, Jinsik Lee, Kyungmin Lee, Sangha Park, Young Min Paik, Yongmin Park, Youngyong Park, Sanghyun Seo, Sihoon Yang, Heuiyeen Yeen, Sihyuk Yi, and Hyeongu Yun. EXAONE 4.0: Unified large language models integrating non-reasoning and reasoning modes. arXiv preprint arXiv:2507.11407, 2025. URL <https://arxiv.org/abs/2507.11407>.
- Loubna Ben Allal, Lewis Tunstall, Anton Lozhkov, Elie Bakouch, Guilherme Penedo, Hynek Kydlicek, and Gabriel Martín Blázquez. Open r1: Update #2. <https://huggingface.co/blog/open-r1/update-2>, February 2025. Accessed: 2026-02-26.
- Amanda Bertsch, Alex Xie, Graham Neubig, and Matthew R. Gormley. It’s MBR all the way down: Modern generation techniques through the lens of minimum Bayes risk. arXiv preprint arXiv:2310.01387, 2023. URL <https://arxiv.org/abs/2310.01387>.
- Bespoke Labs. Bespoke-Stratos: The unreasonable effectiveness of reasoning distillation. <https://www.bespokelabs.ai/blog/bespoke-stratos-the-unreasonable-effectiveness-of-reasoning-distillation>, 2025a. Accessed: 2025-01-22.
- Bespoke Labs. Bespoke-stratos-17k dataset card. <https://huggingface.co/datasets/bespokelabs/Bespoke-Stratos-17k>, 2025b. Accessed: 2025-01-31.
- Bespoke Labs. Bespoke-stratos-32b model card. <https://huggingface.co/bespokelabs/Bespoke-Stratos-32B>, 2025c. Accessed: 2025-01-31.
- Bespoke Labs. Bespoke-stratos-7b model card. <https://huggingface.co/bespokelabs/Bespoke-Stratos-7B>, 2025d. Accessed: 2025-01-31.
- Steven Bethard. We need to talk about random seeds. arXiv preprint arXiv:2210.13393, 2022. URL <https://arxiv.org/abs/2210.13393>.
- Robert E. Blackwell, Jon Barry, and Anthony G. Cohn. Towards reproducible LLM evaluation: Quantifying uncertainty in LLM benchmark scores. arXiv preprint arXiv:2410.03492, 2024. URL <https://arxiv.org/abs/2410.03492>.

-
- Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Dmitriy Serdyuk, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In A. Smola, A. Dimakis, and I. Stoica (eds.), *Proceedings of Machine Learning and Systems*, volume 3, pp. 747–769, 2021. URL https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html.
- Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012. doi: 10.1109/TCIAIG.2012.2186810. URL <https://doi.org/10.1109/TCIAIG.2012.2186810>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. Universal self-consistency for large language model generation. arXiv preprint arXiv:2311.17311, 2023. URL <https://arxiv.org/abs/2311.17311>.
- Julius Cheng and Andreas Vlachos. Faster minimum Bayes risk decoding with confidence-based pruning. arXiv preprint arXiv:2311.14919, 2023. URL <https://arxiv.org/abs/2311.14919>.
- Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot Arena: An open platform for evaluating LLMs by human preference. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 8359–8388. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/chiang24b.html>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, Hao Peng, Yu Cheng, Zhiyuan Liu, Maosong Sun, Bowen Zhou, and Ning Ding. Process reinforcement through implicit rewards. *Transactions on Machine Learning Research*, 2026. URL <https://openreview.net/forum?id=9SkkifLopZ>.
- DeepSeek-AI. deepseek-ai/DeepSeek-R1-Distill-Qwen-32B (model card). <https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B>, 2025a. Accessed: 2026-02-26.
- DeepSeek-AI. DeepSeek-R1 (github repository). <https://github.com/deepseek-ai/DeepSeek-R1>, 2025b. Accessed: 2026-02-26.
- Jasper Dekoninck, Nikola Jovanović, Tim Gehringer, Kári Rognvaldsson, Ivo Petrov, Chenhao Sun, and Martin Vechev. Beyond benchmarks: MathArena as an evaluation platform for mathematics with LLMs. arXiv preprint arXiv:2605.00674, 2026. URL <https://arxiv.org/abs/2605.00674>.

-
- Qiwei Di, Kaixuan Ji, Xuheng Li, Heyang Zhao, and Quanquan Gu. Best-of-Majority: Minimax-optimal strategy for Pass@k inference scaling. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 92146–92171, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/94cbc452f8fe10cc6350e3c06fe75171-Abstract-Conference.html.
- Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552. URL <https://doi.org/10.1214/aos/1176344552>.
- Bryan Eikema and Wilker Aziz. Is MAP decoding all you need? the inadequacy of the mode in neural machine translation. In Donia Scott, Nuria Bel, and Chengqing Zong (eds.), *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 4506–4520, Barcelona, Spain (Online), December 2020. International Committee on Computational Linguistics. doi: 10.18653/v1/2020.coling-main.398. URL <https://aclanthology.org/2020.coling-main.398/>.
- Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In Iryna Gurevych and Yusuke Miyao (eds.), *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 889–898, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1082. URL <https://aclanthology.org/P18-1082/>.
- Markus Freitag, David Grangier, Qijun Tan, and Bowen Liang. High quality rather than high model probability: Minimum Bayes risk decoding with neural metrics. *Transactions of the Association for Computational Linguistics*, 10:811–825, 2022. doi: 10.1162/tacl_a_00491. URL <https://aclanthology.org/2022.tacl-1.47/>.
- Yichao Fu, Xuwei Wang, Hao Zhang, Yuandong Tian, and Jiawei Zhao. Deep think with confidence. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 94355–94377, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/98257285340854262185500e59bc0f28-Abstract-Conference.html.
- FuseAI. Fuseo1-deepseekr1-qwq-skyt1-32b-preview (model card). <https://huggingface.co/FuseAI/FuseO1-DeepSeekR1-QwQ-SkyT1-32B-Preview>, 2025a. Accessed: 2026-02-26.
- FuseAI. Fuseo1-deepseekr1-qwq-skyt1-flash-32b-preview (model card). <https://huggingface.co/FuseAI/FuseO1-DeepSeekR1-QwQ-SkyT1-Flash-32B-Preview>, 2025b. Accessed: 2026-02-26.
- FuseAI. Fuseai: Model fusion for large language models (code and resources). <https://github.com/fanqiwang/FuseAI>, 2025c. Accessed: 2026-02-26.
- GAIR. Gair/limo-v2 (dataset card). <https://huggingface.co/datasets/GAIR/LIMO-v2>, 2025a. Accessed: 2026-02-26.
- GAIR. Gair/limo-v2 (model card). <https://huggingface.co/GAIR/LIMO-v2>, 2025b. Accessed: 2026-02-26.
- Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 10835–10866. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/gao23h.html>.
- Sajjad Ghiasvand, Mark Beliaev, Mahnoosh Alizadeh, and Ramtin Pedarsani. REALM: Reliable expertise-aware language model fine-tuning from noisy annotations. arXiv preprint arXiv:2604.17289, 2026. URL <https://arxiv.org/abs/2604.17289>.
- Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, Ashima Suvarna, Benjamin Feuer, Liangyu Chen, Zaid Khan, Eric Frankel, Sachin Grover, Caroline Choi, Niklas Muennighoff, Shiye Su, Wanjia Zhao, John Yang, Shreyas Pimpalgaonkar, Kartik Sharma, Charlie Cheng-Jie Ji, Yichuan Deng, Sarah Pratt, Vivek Ramanujan, Jon

-
- Saad-Falcon, Jeffrey Li, Achal Dave, Alon Albalak, Kushal Arora, Blake Wulfe, Chinmay Hegde, Greg Durrett, Sewoong Oh, Mohit Bansal, Saadia Gabriel, Aditya Grover, Kai-Wei Chang, Vaishaal Shankar, Aaron Gokaslan, Mike A. Merrill, Tatsunori Hashimoto, Yejin Choi, Jenia Jitsev, Reinhard Heckel, Maheswaran Sathiamoorthy, Alexandros G. Dimakis, and Ludwig Schmidt. Data recipes for reasoning models. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 108059–108130, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/b010241b9f1cdfc7d4c392db899cef86-Abstract-Conference.html.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z. URL <https://doi.org/10.1038/s41586-025-09422-z>.
- Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 8154–8173, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.507. URL <https://aclanthology.org/2023.emnlp-main.507/>.
- Mohsen Hariri, Michael Hinczewski, Jing Ma, and Vipin Chaudhary. Ranking reasoning LLMs under test-time scaling. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 33437–33478, San Diego, California, United States, July 2026a. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.1544. URL <https://aclanthology.org/2026.acl-long.1544/>.
- Mohsen Hariri, Amirhossein Samandar, Michael Hinczewski, and Vipin Chaudhary. Don’t Pass@k: A Bayesian framework for large language model evaluation. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 148539–148579, 2026b. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/f04edfc65463d020629673a4bc4c58e7-Abstract-Conference.html.
- John Hewitt, Christopher D. Manning, and Percy Liang. Truncation sampling as language model desmoothing. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Findings of the Association for*

-
- Computational Linguistics: EMNLP 2022*, pp. 3414–3427, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-emnlp.249. URL <https://aclanthology.org/2022.findings-emnlp.249/>.
- Andreas Hochlehnert, Hardik Bhatnagar, Vishaal Udandara, Samuel Albanie, Ameya Prabhu, and Matthias Bethge. A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=90UrTTxp50>.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rygGQyrFvH>.
- Audrey Huang, Adam Block, Qinghua Liu, Nan Jiang, Akshay Krishnamurthy, and Dylan J Foster. Is Best-of-N the best of them? Coverage, scaling, and optimality in inference-time alignment. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 25075–25126. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/huang25c.html>.
- Hugging Face. Open-R1: A fully open reproduction of DeepSeek-R1. <https://github.com/huggingface/open-r1>, 2025. Accessed: 2026-02-26.
- Chia-Yu Hung, Navonil Majumder, Ambuj Mehrish, and Soujanya Poria. Reward-guided tree search for inference time alignment of large language models. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 12575–12593, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.625. URL <https://aclanthology.org/2025.naacl-long.625/>.
- Minghui Jia. AIME 2024 dataset. Hugging Face dataset, 2024. URL https://huggingface.co/datasets/Maxwell-Jia/AIME_2024.
- Yuu Jinnai and Kaito Ariu. Hyperparameter-free approach for faster minimum Bayes risk decoding. arXiv preprint arXiv:2401.02749, 2024. URL <https://arxiv.org/abs/2401.02749>.
- Yuu Jinnai, Tetsuro Morimura, Kaito Ariu, and Kenshi Abe. Regularized Best-of-N sampling with minimum Bayes risk objective for language model alignment. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 9321–9347, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.472. URL <https://aclanthology.org/2025.naacl-long.472/>.
- Zhewei Kang, Xuandong Zhao, and Dawn Song. Scalable Best-of-N selection for large language models via Self-Certainty. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (eds.), *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pp. 19720–19745. Curran Associates, Inc., 2025. doi: 10.52202/085713-0663. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/1c7eff166a8e345f664f0faa8f4e4d2e-Abstract-Conference.html.
- Hadi Khalaf, Claudio Mayrink Verdun, Alex Oesterling, Himabindu Lakkaraju, and Flavio du Pin Calmon. Inference-time reward hacking in large language models. arXiv preprint arXiv:2506.19248, 2025. URL <https://arxiv.org/abs/2506.19248>.
- Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou (eds.), *Machine Learning: ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, pp. 282–293. Springer Berlin Heidelberg, 2006. ISBN 978-3-540-46056-5. doi: 10.1007/11871842_29. URL https://doi.org/10.1007/11871842_29.

-
- Jing Yu Koh, Stephen Marcus McAleer, Daniel Fried, and Ruslan Salakhutdinov. Tree search for language model agents. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=QF0N3x2XVm>.
- Peng Kuang, Yanli Wang, Xiaoyu Han, Yaowenqi Liu, Kaidi Xu, and Haohan Wang. Optimal aggregation of LLM and PRM signals for efficient test-time scaling. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 92094–92109, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/94bd1b8c906494bada774096bd0fdd73-Abstract-Conference.html.
- Shankar Kumar and William Byrne. Minimum Bayes-risk decoding for statistical machine translation. In *Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics: HLT-NAACL 2004*, pp. 169–176, Boston, Massachusetts, USA, 2–7 May 2004. Association for Computational Linguistics. URL <https://aclanthology.org/N04-1022/>.
- Jacky Kwok, Shulu Li, Pranav Atreya, Yuejiang Liu, Yixing Jiang, Chelsea Finn, Marco Pavone, Ion Stoica, and Azalia Mirhoseini. LLM-as-a-Verifier: A general-purpose verification framework. arXiv preprint arXiv:2607.05391, 2026. URL <https://arxiv.org/abs/2607.05391>.
- Dacheng Li, Shiyi Cao, Tyler Griggs, Shu Liu, Xiangxi Mo, Eric Tang, Sumanth Hegde, Kourosh Hakhmaneshi, Shishir G. Patil, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Language models can easily learn to reason from demonstrations. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 15979–15997, Suzhou, China, November 2025a. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.866. URL <https://aclanthology.org/2025.findings-emnlp.866/>.
- Xuefeng Li, Haoyang Zou, and Pengfei Liu. LIMR: Less is more for RL scaling. arXiv preprint arXiv:2502.11886, 2025b. URL <https://arxiv.org/abs/2502.11886>.
- Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. Escape sky-high cost: Early-stopping Self-Consistency for multi-step reasoning. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (eds.), *International Conference on Learning Representations*, volume 2024, pp. 14751–14768, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/3fe2a777282299ecb4f9e7ebb531f0ab-Abstract-Conference.html.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158. URL <https://doi.org/10.1126/science.abq1158>.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Alexander Cosgrove, Christopher D Manning, Christopher Re, Diana Acosta-Navas, Drew Arad Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue WANG, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri S. Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Andrew Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=i04LZibEqW>. Featured Certification, Expert Certification.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In B. Kim, Y. Yue, S. Chaudhuri,

-
- K. Fragkiadaki, M. Khan, and Y. Sun (eds.), *International Conference on Learning Representations*, volume 2024, pp. 39578–39601, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/aca97732e30bcf1303bc22ac3924fd16-Abstract-Conference.html.
- Junnan Liu, Hongwei Liu, Linchen Xiao, Ziyi Wang, Kuikun Liu, Songyang Gao, Wenwei Zhang, Songyang Zhang, and Kai Chen. Are your LLMs capable of stable reasoning? In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 17594–17632, Vienna, Austria, July 2025a. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.905. URL <https://aclanthology.org/2025.findings-acl.905/>.
- Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. Quantization hurts reasoning? an empirical study on quantized reasoning models. In *Second Conference on Language Modeling*, 2025b. URL <https://openreview.net/forum?id=BM192Ps5Nv>.
- Shudong Liu, Hongwei Liu, Junnan Liu, Linchen Xiao, Songyang Gao, Chengqi Lyu, Yuzhe Gu, Wenwei Zhang, Derek F. Wong, Songyang Zhang, and Kai Chen. CompassVerifier: A unified and robust verifier for LLMs evaluation and outcome reward. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33466–33494, Suzhou, China, November 2025c. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1698. URL <https://aclanthology.org/2025.emnlp-main.1698/>.
- Zihan Liu, Zhuolin Yang, Yang Chen, Chankyu Lee, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. AceReason-Nemotron 1.1: Advancing math and code reasoning through SFT and RL synergy. arXiv preprint arXiv:2506.13284, 2025d. URL <https://arxiv.org/abs/2506.13284>.
- M-A-P, ByteDance Seed, and 2077AI. SuperGPQA: Scaling LLM evaluation across 285 graduate disciplines. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (eds.), *Advances in Neural Information Processing Systems*, volume 38, Main Conference. Curran Associates, Inc., 2025. doi: 10.52202/085713-3766. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/a3c5af1f56fc73eef1ba0f442739f5ca-Abstract-Datasets_and_Benchmarks_Track.html.
- M-A-P Team. SuperGPQA dataset card. Hugging Face dataset, 2025. URL <https://huggingface.co/datasets/m-a-p/SuperGPQA>.
- MathArena. BRUMO 2025 dataset card. Hugging Face dataset, 2025a. URL https://huggingface.co/datasets/MathArena/brumo_2025.
- MathArena. CMIMC 2025 dataset card. Hugging Face dataset, 2025b. URL https://huggingface.co/datasets/MathArena/cmimc_2025.
- MathArena. HMMT February 2025 dataset card. Hugging Face dataset, 2025c. URL https://huggingface.co/datasets/MathArena/hmmt_feb_2025.
- MathArena. HMMT November 2025 dataset card. Hugging Face dataset, 2025d. URL https://huggingface.co/datasets/MathArena/hmmt_nov_2025.
- MathArena. SMT 2025 dataset card. Hugging Face dataset, 2025e. URL https://huggingface.co/datasets/MathArena/smt_2025.
- MathArena. AIME 2026 dataset card. Hugging Face dataset, 2026a. URL https://huggingface.co/datasets/MathArena/aime_2026.
- MathArena. HMMT February 2026 dataset card. Hugging Face dataset, 2026b. URL https://huggingface.co/datasets/MathArena/hmmt_feb_2026.
- Clara Meister, Tiago Pimentel, Gian Wiher, and Ryan Cotterell. Locally typical sampling. *Transactions of the Association for Computational Linguistics*, 11:102–121, 2023. doi: 10.1162/tac1_a_00536. URL <https://aclanthology.org/2023.tac1-1.7/>.

-
- Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple preference optimization with a reference-free reward. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 124198–124235. Curran Associates, Inc., 2024. doi: 10.52202/079017-3946. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/e099c1c9699814af0be873a175361713-Abstract-Conference.html.
- Microsoft. microsoft/phi-4-mini-flash-reasoning (model card). <https://huggingface.co/microsoft/Phi-4-mini-flash-reasoning>, 2025. Accessed: 2026-02-26.
- Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. arXiv preprint arXiv:2411.00640, 2024. URL <https://arxiv.org/abs/2411.00640>.
- ModelScope Team. EvalScope: Evaluation framework for large models, 2024. URL <https://github.com/modelscope/evalscope>.
- Ivan Moshkov, Darragh Hanley, Ivan Sorokin, Shubham Toshniwal, Christof Henkel, Benedikt Schifferer, Wei Du, and Igor Gitman. AIMO-2 winning solution: Building state-of-the-art mathematical reasoning models with OpenMathReasoning dataset. arXiv preprint arXiv:2504.16891, 2025. URL <https://arxiv.org/abs/2504.16891>.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 20275–20321, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1025. URL <https://aclanthology.org/2025.emnlp-main.1025/>.
- NovaSky Team. Think less, achieve more: Cut reasoning costs by 50% without sacrificing accuracy. <https://novasky-ai.github.io/posts/reduce-overthinking>, 2025a. Accessed: 2025-01-23.
- NovaSky Team. Sky-T1: Train your own O1 preview model within \$450. <https://novasky-ai.github.io/posts/sky-t1>, 2025b. Accessed: 2025-01-09.
- NovaSky Team. Unlocking the potential of reinforcement learning in improving reasoning models. <https://novasky-ai.github.io/posts/sky-t1-7B/>, 2025c. Accessed: 2025-02-13.
- NVIDIA. OpenReasoning-Nemotron-1.5B model card. Hugging Face model card, 2025. URL <https://huggingface.co/nvidia/OpenReasoning-Nemotron-1.5B>. Released: 2025-07-16; Accessed: 2026-08-29.
- NVIDIA, Aarti Basant, Abhijit Khairnar, Abhijit Paithankar, Abhinav Khattar, Adithya Renduchintala, Aditya Malte, Akhiad Bercovich, Akshay Hazare, Alejandra Rico, Aleksander Ficek, Alex Kondratenko, Alex Shaposhnikov, Alexander Bukharin, Ali Taghibakhshi, Amelia Barton, Ameya Sunil Mahabaleshwar, Amy Shen, Andrew Tao, Ann Guan, Anna Shors, Anubhav Mandarwal, Arham Mehta, Arun Venkatesan, Ashton Sharabiani, Ashwath Aithal, Ashwin Poojary, Ayush Dattagupta, Balaram Budharaju, Banghua Zhu, Barnaby Simkin, Bilal Kartal, Bitar Darvish Rouhani, Bobby Chen, Boris Ginsburg, Brandon Norick, Brian Yu, Bryan Catanzaro, Charles Wang, Charlie Truong, Chetan Mungekar, Chintan Patel, Chris Alexiuk, Christian Munley, Christopher Parisien, Dan Su, Daniel Afrimi, Daniel Korzekwa, Daniel Rohrer, Daria Gitman, David Mosallanezhad, Deepak Narayanan, Dima Rekish, Dina Yared, Dmytro Pykhtar, Dong Ahn, Duncan Riach, Eileen Long, Elliott Ning, Eric Chung, Erick Galinkin, Evelina Bakhturina, Gargi Prasad, Gerald Shen, Haifeng Qian, Haim Elisha, Harsh Sharma, Hayley Ross, Helen Ngo, Herman Sahota, Hexin Wang, Hoo Chang Shin, Hua Huang, Iain Cunningham, Igor Gitman, Ivan Moshkov, Jaehun Jung, Jan Kautz, Jane Polak Scowcroft, Jared Casper, Jian Zhang, Jiaqi Zeng, Jimmy Zhang, Jinze Xue, Jocelyn Huang, Joey Conway, John Kamalu, Jonathan Cohen, Joseph Jennings, Julien Veron Vialard, Junkeun Yi, Jupinder Parmar, Kari Briski, Katherine Cheung, Katherine Luna, Keith Wyss, Keshav Santhanam, Kezhi Kong, Krzysztof Pawelec, Kumar Anik, Kunlun Li, Kushan Ahmadian, Lawrence McAfee, Laya Sleiman, Leon Derczynski, Luis Vega, Maer Rodrigues

de Melo, Makesh Narsimhan Sreedhar, Marcin Chochowski, Mark Cai, Markus Kliegl, Marta Stepniewska-Dziubinska, Matvei Novikov, Mehrzad Samadi, Meredith Price, Meriem Boubdir, Michael Boone, Michael Evans, Michal Bien, Michal Zawalski, Miguel Martinez, Mike Chrzanowski, Mohammad Shoeybi, Mostofa Patwary, Namit Dhameja, Nave Assaf, Negar Habibi, Nidhi Bhatia, Nikki Pope, Nima Tajbakhsh, Nirmal Kumar Juluru, Oleg Rybakov, Oleksii Hrinchuk, Oleksii Kuchaiev, Oluwatobi Olabiyi, Pablo Ribalta, Padmavathy Subramanian, Parth Chadha, Pavlo Molchanov, Peter Dykas, Peter Jin, Piotr Bialecki, Piotr Januszewski, Pradeep Thalasta, Prashant Gaikwad, Prasoon Varshney, Pritam Gundecha, Przemek Tredak, Rabeeh Karimi Mahabadi, Rajen Patel, Ran El-Yaniv, Ranjit Rajan, Ria Cheruvu, Rima Shahbazyan, Ritika Borkar, Ritu Gala, Roger Waleffe, Ruoxi Zhang, Russell J. Hewett, Ryan Prenger, Sahil Jain, Samuel Krizan, Sanjeev Satheesh, Saori Kaji, Sarah Yurick, Saurav Muralidharan, Sean Narenthiran, Seonmyeong Bak, Sepehr Sameni, Seungju Han, Shanmugam Ramasamy, Shaona Ghosh, Sharath Turuvekere Sreenivas, Shelby Thomas, Shizhe Diao, Shreya Gopal, Shrimai Prabhumoye, Shubham Toshniwal, Shuoyang Ding, Siddharth Singh, Siddhartha Jain, Somshubra Majumdar, Soumye Singhal, Stefania Alborghetti, Syeda Nahida Akter, Terry Kong, Tim Moon, Tomasz Hliwiak, Tomer Asida, Tony Wang, Tugrul Konuk, Twinkle Vashishth, Tyler Poon, Udi Karpas, Vahid Noroozi, Venkat Srinivasan, Vijay Korthikanti, Vikram Fugro, Vineeth Kalluru, Vitaly Kurin, Vitaly Lavrukhin, Wasi Uddin Ahmad, Wei Du, Wonmin Byeon, Ximing Lu, Xin Dong, Yashaswi Karnati, Yejin Choi, Yian Zhang, Ying Lin, Yonggan Fu, Yoshi Suhara, Zhen Dong, Zhiyu Li, Zhongbo Zhu, and Zijia Chen. NVIDIA Nemotron Nano 2: An accurate and efficient hybrid Mamba-Transformer reasoning model. arXiv preprint arXiv:2508.14444, 2025. URL <https://arxiv.org/abs/2508.14444>.

Open-R1 Team. CodeForces-CoTs dataset card. <https://huggingface.co/datasets/open-r1/codeforces-cots>, 2025a. Accessed: 2026-02-26.

Open-R1 Team. Mixture-of-thoughts dataset card. <https://huggingface.co/datasets/open-r1/Mixture-of-Thoughts>, 2025b. Accessed: 2026-02-26.

Open-R1 Team. Olympiccoder-32b model card. <https://huggingface.co/open-r1/OlympicCoder-32B>, 2025c. Accessed: 2026-02-26.

Open-R1 Team. Olympiccoder-7b model card. <https://huggingface.co/open-r1/OlympicCoder-7B>, 2025d. Accessed: 2026-02-26.

Open-R1 Team. Openr1-distill-7b model card. <https://huggingface.co/open-r1/OpenR1-Distill-7B>, 2025e. Accessed: 2026-02-26.

Open-R1 Team. Openr1-math-220k dataset card. <https://huggingface.co/datasets/open-r1/OpenR1-Math-220k>, 2025f. Accessed: 2026-02-26.

OpenAI. gpt-oss-20b model (openai api documentation). <https://developers.openai.com/api/docs/models/gpt-oss-20b>, 2025a. Accessed: 2026-02-26.

OpenAI. openai/gpt-oss-20b (model card). <https://huggingface.co/openai/gpt-oss-20b>, 2025b. Accessed: 2026-02-26.

OpenAI, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrylov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado, Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily (Xiaoxuan) Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry

Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. gpt-oss-120b & gpt-oss-20b model card. arXiv preprint arXiv:2508.10925, 2025. URL <https://arxiv.org/abs/2508.10925>.

OpenThoughts. Launching the OpenThoughts project. <https://open-thoughts.ai/blog/launch>, January 2025.

OpenThoughts Team. Openthinker2-32b model card. <https://huggingface.co/open-thoughts/OpenThinker2-32B>, 2025a. Accessed: 2026-02-26.

OpenThoughts Team. Openthinker3-7b model card. <https://huggingface.co/open-thoughts/OpenThinker3-7B>, 2025b. Accessed: 2026-02-26.

OpenThoughts Team. Openthoughts-114k dataset card. <https://huggingface.co/datasets/open-thoughts/OpenThoughts-114k>, 2025c. Accessed: 2026-02-26.

OpenThoughts Team. Openthoughts2-1m dataset card. <https://huggingface.co/datasets/open-thoughts/OpenThoughts2-1M>, 2025d. Accessed: 2026-02-26.

OpenThoughts Team. Openthoughts3-1.2m dataset card. <https://huggingface.co/datasets/open-thoughts/OpenThoughts3-1.2M>, 2025e. Accessed: 2026-02-26.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 27730–27744. Curran Associates, Inc., 2022. doi: 10.52202/068431-2011. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract.html.

Guilherme Penedo, Lewis Tunstall, Anton Lozhkov, Hynek Kydlicek, Edward Beeching, Loubna Ben Allal, Quentin Gallouédec, Leandro von Werra, Agustín Piqueres Lajarín, and Nathan Habib. Open r1: Update #3. <https://huggingface.co/blog/open-r1/update-3>, March 2025. Accessed: 2026-02-26.

Qihoo360. qihoo360/light-r1-14b-ds (model card). <https://huggingface.co/qihoo360/Light-R1-14B-DS>, 2025a. Accessed: 2026-02-26.

Qihoo360. qihoo360/light-r1-32b (model card). <https://huggingface.co/qihoo360/Light-R1-32B>, 2025b. Accessed: 2026-02-26.

Qihoo360. qihoo360/light-r1-32b-ds (model card). <https://huggingface.co/qihoo360/Light-R1-32B-DS>, 2025c. Accessed: 2026-02-26.

Qihoo360. qihoo360/light-r1-7b-ds (model card). <https://huggingface.co/qihoo360/Light-R1-7B-DS>, 2025d. Accessed: 2026-02-26.

Qwen Team. Qwen/QwQ-32B (model card). <https://huggingface.co/Qwen/QwQ-32B>, 2025a. Accessed: 2026-02-26.

Qwen Team. Qwen/qwen3-30b-a3b-thinking-2507 (model card). <https://huggingface.co/Qwen/Qwen3-30B-A3B-Thinking-2507>, 2025b. Accessed: 2026-02-26.

Qwen Team. Qwen/qwen3-4b-thinking-2507 (model card). <https://huggingface.co/Qwen/Qwen3-4B-Thinking-2507>, 2025c. Accessed: 2026-02-26.

-
- Qwen Team. Qwen/qwen3-next-80b-a3b-thinking (model card). <https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Thinking>, 2025d. Accessed: 2026-02-26.
- Qwen Team. QwQ-32B: Embracing the power of reinforcement learning, March 2025e. URL <https://qwenlm.github.io/blog/qwq-32b/>.
- Qwen Team. Qwen/qwen3.6-35b-a3b (model card). <https://huggingface.co/Qwen/Qwen3.6-35B-A3B>, 2026. Accessed: 2026-08-31.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 53728–53741. Curran Associates, Inc., 2023. doi: 10.52202/075280-2338. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8302b5e06ce7-Abstract-Conference.html.
- Amin Rakhsha, Kanika Madan, Tianyu Zhang, Amir-massoud Farahmand, and Amir Khasahmadi. Majority of the bests: Improving Best-of-N via bootstrapping. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (eds.), *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pp. 37844–37877. Curran Associates, Inc., 2025. doi: 10.52202/085713-1268. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/36556567e8437f137da23047309155dd-Abstract-Conference.html.
- Liliang Ren, Congcong Chen, Haoran Xu, Young Jin Kim, Adam Atkinson, Zheng Zhan, Jiankai Sun, Baolin Peng, Liyuan Liu, Shuohang Wang, Hao Cheng, Jianfeng Gao, Weizhu Chen, and Yelong Shen. Decoder-hybrid-decoder architecture for efficient reasoning with long generation. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (eds.), *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pp. 107646–107680. Curran Associates, Inc., 2025. doi: 10.52202/085713-3591. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/9aa51796f8bede2ea947d6b6e3087ab8-Abstract-Conference.html.
- SaylorTwift. BIG-Bench Hard dataset card. Hugging Face dataset mirror, 2024. URL <https://huggingface.co/datasets/SaylorTwift/bbh>.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 10131–10165, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b623663fd9b874366f3ce019fd9dd44-Abstract-Conference.html.
- Lin Sun, Guangxiang Zhao, Xiaoqi Jian, Yuhang Wu, Weihong Lin, Yongfu Zhu, Qilong Shi, Chang Jia, Aomufei Yuan, Yuxuan Tian, Linglin Zhang, Jinzhu Wu, Junfeng Ran, Sai-er Hu, Zihan Jiang, Juntong Zhou, Wenrui Liu, Xusen Xiao, Bin Cui, Tong Yang, and Xiangzheng Zhang. TinyR1-32B-Preview: Boosting accuracy with branch-merge distillation. arXiv preprint arXiv:2503.04872, 2025. URL <https://arxiv.org/abs/2503.04872>.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 13003–13051, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.824. URL <https://aclanthology.org/2023.findings-acl.824/>.
- Ahmad Tahmasivand, Noureldin Zahran, Saba Al-Sayouri, Mohammed Fouda, and Khaled N. Khasawneh. Lm-fix: Lightweight bit-flip detection and rapid recovery framework for language models. In *2025 IEEE 43rd International Conference on Computer Design (ICCD)*, 2025. doi: 10.1109/ICCD65941.2025.00068.

-
- Amir Taubenfeld, Tom Sheffer, Eran Ofek, Amir Feder, Ariel Goldstein, Zorik Gekhman, and Gal Yona. Confidence improves self-consistency in LLMs. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 20090–20111, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.1030. URL <https://aclanthology.org/2025.findings-acl.1030/>.
- TIGER-Lab. MMLU-Pro dataset card. Hugging Face dataset, 2024. URL <https://huggingface.co/datasets/TIGER-Lab/MMLU-Pro>.
- Claudio Mayrink Verdun, Alex Oesterling, Himabindu Lakkaraju, and Flavio P. Calmon. Soft best-of- n sampling for model alignment. arXiv preprint arXiv:2505.03156, 2025. URL <https://arxiv.org/abs/2505.03156>.
- Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (eds.), *International Conference on Learning Representations*, volume 2024, pp. 18303–18322, 2024a. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/4fd5cfd2e31bebbccfa5ffa354c04bdc-Abstract-Conference.html.
- Fanqi Wan, Longguang Zhong, Ziyi Yang, Ruijun Chen, and Xiaojun Quan. FuseChat: Knowledge fusion of chat models. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 21618–21642, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1096. URL <https://aclanthology.org/2025.emnlp-main.1096/>.
- Ziyu Wan, Xidong Feng, Muning Wen, Stephen Marcus McAleer, Ying Wen, Weinan Zhang, and Jun Wang. AlphaZero-like tree-search can guide large language model decoding and training. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 49890–49920. PMLR, 21–27 Jul 2024b. URL <https://proceedings.mlr.press/v235/wan24c.html>.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 95266–95290. Curran Associates, Inc., 2024. doi: 10.52202/079017-3018. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ad236edc564f3e3156e1b2feafb99a24-Abstract-Datasets_and_Benchmarks_Track.html.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 24824–24837. Curran Associates, Inc., 2022. doi: 10.52202/068431-1800. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract.html.
- Sean Welleck, Amanda Bertsch, Matthew Finlayson, Hailey Schoelkopf, Alex Xie, Graham Neubig, Ilya Kulikov, and Zaid Harchaoui. From decoding to meta-generation: Inference-time algorithms for large language models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=eskQMCIbMS>. Survey Certification.

-
- Liang Wen, Yunke Cai, Fenrui Xiao, Xin He, Qi An, Zhenyu Duan, Yimin Du, Junchen Liu, Lifu Tang, Xiaowei Lv, Haosheng Zou, Yongchao Deng, Shousheng Jia, and Xiangzheng Zhang. Light-R1: Curriculum SFT, DPO and RL for long COT from scratch and beyond. In Georg Rehm and Yunyao Li (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pp. 318–327, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-288-6. doi: 10.18653/v1/2025.acl-industry.24. URL <https://aclanthology.org/2025.acl-industry.24/>.
- Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. arXiv preprint arXiv:2408.00724, 2024. URL <https://arxiv.org/abs/2408.00724>.
- Haoran Xu, Baolin Peng, Hany Awadalla, Dongdong Chen, Yen-Chun Chen, Mei Gao, Young Jin Kim, Yunsheng Li, Liliang Ren, Yelong Shen, Shuohang Wang, Weijian Xu, Jianfeng Gao, and Weizhu Chen. Phi-4-Mini-Reasoning: Exploring the limits of small reasoning language models in math. arXiv preprint arXiv:2504.21233, 2025. URL <https://arxiv.org/abs/2504.21233>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025a. URL <https://arxiv.org/abs/2505.09388>.
- Ziyi Yang, Fanqi Wan, Longguang Zhong, Canbin Huang, Guosheng Liang, and Xiaojun Quan. FuseChat-3.0: Preference optimization meets heterogeneous model fusion. arXiv preprint arXiv:2503.04222, 2025b. URL <https://arxiv.org/abs/2503.04222>.
- Jinwei Yao, Kaiqi Chen, Kexun Zhang, Jiaxuan You, Binhang Yuan, Zeke Wang, and Tao Lin. DeFT: Decoding with Flash tree-attention for efficient tree-structured LLM inference. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 66657–66688, 2025a. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/a6df53f082619d02b9fad64a022e5de3-Abstract-Conference.html.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 11809–11822. Curran Associates, Inc., 2023. doi: 10.52202/075280-0517. URL <https://proceedings.neurips.cc/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract.html>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for Tool-Agent-User interaction in real-world domains. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 9965–10017, 2025b. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.
- Fanghua Ye, Mingming Yang, Jianhui Pang, Longyue Wang, Derek Wong, Emine Yilmaz, Shuming Shi, and Zhaopeng Tu. Benchmarking LLMs via uncertainty quantification. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 15356–15385. Curran Associates, Inc., 2024. doi: 10.52202/079017-0491. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/1bdc065d40203a00bd39831153338bb-Abstract-Datasets_and_Benchmarks_Track.html.

-
- Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. LIMO: Less is more for reasoning. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=T2TZ0RY4Zk>.
- Fei Yu, Anningzhe Gao, and Benyou Wang. OVM, outcome-supervised value models for planning in mathematical reasoning. In Kevin Duh, Helena Gomez, and Steven Bethard (eds.), *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 858–875, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-naacl.55. URL <https://aclanthology.org/2024.findings-naacl.55/>.
- Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (eds.), *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pp. 169819–169851. Curran Associates, Inc., 2025. doi: 10.52202/085713-5653. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/f80094a824ba5912d4a2de169c404a40-Abstract-Conference.html.
- Yifan Zhang and Team Math-AI. American invitational mathematics examination (AIME) 2025. Hugging Face dataset, 2025. URL <https://huggingface.co/datasets/math-ai/aime25>.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. arXiv preprint arXiv:2507.18071, 2025. URL <https://arxiv.org/abs/2507.18071>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv preprint arXiv:2306.05685, 2023. URL <https://arxiv.org/abs/2306.05685>.
- Yilun Zhou, Austin Xu, Peifeng Wang, Caiming Xiong, and Shafiq Joty. Evaluating judges as evaluators: The JETTS benchmark of LLM-as-judges as test-time scaling evaluators. arXiv preprint arXiv:2504.15253, 2025. URL <https://arxiv.org/abs/2504.15253>.

A Notation

The formalization (Section 2), evaluation framework (Section 3), and leaf- and prefix-level variants in Appendices B and C use a common notation.

Core objects and budgeted inference.

Symbol	Meaning
$x \in \mathcal{X}$	Input prompt or problem instance.
$p_\theta(\cdot x)$	Base autoregressive language model.
$\mathcal{T}(x)$	Implicit rooted prefix tree induced by generation from x .
$\mathcal{L}(x)$	Terminal leaves of $\mathcal{T}(x)$, i.e., completed generations.
$z \preceq y$	Prefix relation: search state or prefix z lies on the path to completed leaf y .
π	Local generation policy or decoder used to extend a trajectory.
$q_\pi(y x)$	Proposal distribution over completed leaves induced by p_θ together with policy π .
$\mathcal{A}_B$	Budgeted test-time algorithm with total cost at most B .
$\mathcal{B}$	Declared set of allowed nonnegative budgets.
$o_t, c(o_t)$	Primitive operation at step t and its associated cost.
B, b_t	Total test-time budget and remaining budget at step t .
$\mathcal{O}(x)$	Disjoint union of completed-leaf and answer-valued output spaces.
$U_x(o)$	Task utility of output $o \in \mathcal{O}(x)$ on instance x .
$\hat{o}_B(x)$	Unified output returned by $\mathcal{A}_B$; written $\hat{y}_B(x)$ or $\hat{a}_B(x)$ according to its type.
ξ	Internal (algorithmic) randomness of $\mathcal{A}_B$; ξ_q is its realization on prompt x_q .
$G(B)$	Test-time scaling curve $\mathbb{E}_{x \sim \mathcal{P}, \xi} [U_x(\hat{o}_B(x; \xi))]$.
π_{seq}, ν_t	Single-trajectory controller and its meta-action at step t (distinct from the correct-candidate count ν_q).

Output interpretation and evaluation signals.

Symbol	Meaning
$\text{Parse}(y) = (r, a)$	Deterministic parser that extracts reasoning trace r and answer a from completed generation y .
$\Psi(y) = (d, s)$	Decomposition of y into a deterministically verifiable component d and a non-verifiable remainder s .
$V_P(x, d)$	Programmatic verifier applied to instance x and verifiable artifact d .
$\mathcal{V}$	Codomain of verifier outputs, which may be Boolean, graded, diagnostic, or canonicalized.
PASS	Subset of verifier outputs treated as acceptable by a reducer.
$J_\phi(x, \omega)$	Learned evaluator on object ω , where ω may be a completed leaf y or a partial state z .

Decoding protocols and induced proposals.

Symbol	Meaning
Σ	Token vocabulary.
h_t, ℓ_t, p_t	Partial generation $(x, y_{<t})$, its logits, and the softmax token probabilities.
$D_\lambda, \sigma_t, \chi_t$	One-token decoder with configuration λ , its finite state, and side information.
u_t, S_t	Nonnegative token measure and retained support of the energy-gate decoder (distinct from the MBR utility $u(\cdot, \cdot)$ and the prefix score $S(z)$).
$\tilde{q}_t$	One-token proposal $\text{Norm}_{S_t}(u_t)$.
$q_\lambda(y x)$	Sequence-level proposal induced by composing the local kernels; written q_π when the protocol is denoted π .

Evaluation under test-time scaling.

Symbol	Meaning
$\mathcal{P}$	Task distribution over problem instances.
$\mathcal{D}_{\text{eval}} = \{x_q\}_{q=1}^Q$	Evaluation set with Q prompts or problem instances.
$M_B, \hat{M}_B$	Population and empirical end-to-end performance of algorithm $\mathcal{A}_B$ under budget B .
$Z_{q,i}$	Binary correctness indicator for candidate i on prompt q inside a sampled bank.
ν_q	Number of correct candidates in the bank for prompt q , i.e., $\sum_{i=1}^N Z_{q,i}$.
p_q	Latent single-attempt success probability for prompt q under the fixed proposal.
k, t	Attempt budget of a repeated-sampling diagnostic and success-count threshold, $1 \leq t \leq k$; finite-bank versions require $k \leq N$.
$\kappa_{k,t}(p)$	Binomial tail kernel $\Pr\{\text{Binomial}(k, p) \geq t\}$.
$S_{k,t}$	Discovery–stability profile coordinate: expected fraction of prompts with at least t correct candidates among k fresh attempts.
$\hat{S}_{k,t}^{\text{bank}}$	Finite-bank (without-replacement) estimate of the same threshold event computed from an N -candidate bank.
$\mathcal{H}_n, \mathcal{F}_{i,t}$	Information available after n completed rollouts and through token t of rollout i , respectively.
$\tau_{\text{roll}}, \tau_{\text{tok}}$	Across-rollout and within-rollout stopping times.

Leaf-level scaling.

Symbol	Meaning
N	Number of completed candidates in a leaf bank.
$[N]$	Candidate-index set $\{1, \dots, N\}$.
$\mathcal{Y}_N(x) = \{Y_i\}_{i=1}^N$	Multiset of completed candidates generated for input x .
Y_i	i th completed candidate (leaf).
a_i, m_i	Canonical answer parsed from Y_i and its inference-time signals; $a_i = \perp$ denotes an invalid candidate when applicable.
$\mathcal{I}_{\text{valid}}$	Indices of reducer-eligible candidates, excluding $a_i = \perp$ for answer-valued tasks.
v_i, j_i	Verifier output $V_P(x, d_i)$ and learned score $J_\phi(x, Y_i)$ for candidate Y_i .
$E_\eta, \mathcal{E}, e_i$	Evidence map, evidence space, and candidate evidence object $e_i = E_\eta(x, Y_i, m_i)$; e_i is scalar for score-based rules.
$\mathbf{W}_n$	Pairwise utility matrix over the first n completed candidates.
$\mathcal{G}_\delta, \hat{o}_n$	Fixed-bank decision rule and its returned leaf, answer, or output set.
$\mathcal{R}_N$	Leaf-level reducer that maps a candidate bank to a final leaf or answer.
$T(x, Y_i, v_i, j_i)$	Tie-breaking score used by verifier-constrained selection.
i^*	Index of the selected candidate when the reducer returns one sampled leaf.
$\hat{q}_N(a \mid x)$	Empirical answer distribution induced by the leaf bank.
$\psi(\cdot)$	Deterministic projection of a leaf used inside an MBR objective.
$u(\cdot, \cdot)$	Utility or similarity function used by an MBR-style reducer.
$C_{\text{gen}}(n), C_{\text{eval}}(n)$	Generation cost and inference-time scoring, control, and reduction cost for a bank of size n .
$C_{\text{signal}}, C_{\text{control}}, C_{\text{decision}}, C_{\text{total}}$	Evidence-acquisition, controller, decision, and total protocol cost.

Prefix-level scaling.

Symbol	Meaning
z	Search state: a literal token prefix or a macro-prefix consisting of one or more reasoning steps.
$\text{Succ}(z)$	Allowed successor states obtained by expanding prefix z .
$\mathcal{F}_t$	Active frontier of unfinished search states at step t .
$\mathcal{Y}_t$	Bank of completed leaves accumulated by a prefix search up to step t .
ρ	Rollout policy used to continue a prefix to completion.
$q_\rho(\cdot \mid x, z)$	Distribution over completed leaves obtained by continuing from prefix z with rollout policy ρ .
$Q_\rho^*(z)$	Continuation value of prefix z under rollout policy ρ .
$\hat{Q}(z)$	Approximate continuation-value estimate used for pruning or prioritization.
$S(z)$	Generic prefix score used by the search controller.
$S_{\text{LL}}(z)$	Likelihood-based prefix score.
$R_\phi, S_{\text{PRM}}(z)$	Process reward model (step-level scorer) and the prefix score it induces.
$H(x, z)$	Partial check; sound for rejection when $H(x, z) = 0$ implies $U_x(y) = 0$ for all $y \succeq z$.
π_{ctrl}, a_t	Search controller and its meta-action (expand, generate successors, allocate rollouts, prune, terminate).
$\hat{Q}_{\text{MC}}(z)$	Monte Carlo estimate of continuation value using M_{roll} rollouts from prefix z .
K	Beam width in beam search.
M_{roll}	Number of rollouts used when estimating $\hat{Q}_{\text{MC}}(z)$.
$n(z), n(z, a)$	Visit counts for state z and action a in an MCTS-style search tree.
$q_{\text{search}, B}(y \mid x)$	Effective leaf distribution induced by a prefix-level search algorithm under budget B .
$Q_{\text{search}, B}(\mathcal{Y} \mid x)$	Joint law of an ordered bank or multiset returned by prefix search.

B Leaf-level reduction: additional derivations and variants

Under the leaf-level formalism of Section 2.3, self-consistency is an empirical MBR rule, and weighted candidate banks use the same template. Answer selection and rationale presentation remain separate decisions.

B.1 Self-consistency as empirical MBR

Let $\psi(y)$ be the extracted answer, i.e., $\psi(y) = a$ with $(r, a) = \text{Parse}(y)$, let $\mathcal{I}_{\text{valid}} = \{i \in [N] : a_i \neq \perp\}$, and suppose this set is nonempty. With $u(a, a') = \mathbf{1}[a = a']$, the empirical MBR objective over the eligible leaves of $\mathcal{Y}_N(x) = \{Y_i\}_{i=1}^N$ is

$$\hat{R}_N(i) = \frac{1}{|\mathcal{I}_{\text{valid}}|} \sum_{k \in \mathcal{I}_{\text{valid}}} u(a_i, a_k) = \frac{1}{|\mathcal{I}_{\text{valid}}|} \sum_{k \in \mathcal{I}_{\text{valid}}} \mathbf{1}[a_i = a_k] = \hat{q}_N(a_i \mid x).$$

Therefore,

$$i^* \in \arg \max_{i \in \mathcal{I}_{\text{valid}}} \hat{R}_N(i) \iff \hat{a} \in \arg \max_{a \neq \perp} \hat{q}_N(a \mid x).$$

Plurality self-consistency is thus exactly empirical MBR with a zero-one answer-agreement utility (Wang et al., 2023; Kumar & Byrne, 2004). The declared all-invalid fallback and deterministic tie rule from Section 2.3 complete both definitions.

B.2 Weighted banks and heterogeneous proposals

A weighted variant replaces the uniform empirical distribution by

$$\hat{q}_w(a \mid x) = \frac{\sum_{i \in \mathcal{I}_{\text{valid}}} w_i \mathbf{1}[a_i = a]}{\sum_{i \in \mathcal{I}_{\text{valid}}} w_i}, \quad w_i \geq 0, \quad \sum_{i \in \mathcal{I}_{\text{valid}}} w_i > 0.$$

The corresponding weighted MBR objective is

$$\hat{R}_w(i) = \sum_{k \in \mathcal{I}_{\text{valid}}} \bar{w}_k u(\psi(Y_i), \psi(Y_k)), \quad \bar{w}_k = \frac{w_k}{\sum_{j \in \mathcal{I}_{\text{valid}}} w_j}.$$

The weighting scheme applies to confidence-weighted consensus and to banks built from heterogeneous proposal distributions. If candidate Y_i is drawn from proposal $q_i(\cdot | x)$ but the desired risk is defined under target proposal $q_\star(\cdot | x)$, assume $q_\star(\cdot | x)$ is absolutely continuous with respect to every contributing $q_i(\cdot | x)$. Then importance weights

$$w_i \propto \frac{q_\star(Y_i | x)}{q_i(Y_i | x)}$$

define a self-normalized importance estimator when the probability ratios are available. If truncated decoding or adaptive search prevents their evaluation, replacement weights are heuristic rather than an exact correction.

B.3 Beyond exact-match consensus

Open-ended outputs may not admit a canonical exact-match representation. A reducer can instead use a utility or similarity kernel over projected candidates,

$$s_i = \sum_{\ell=1}^N u(\psi(Y_i), \psi(Y_\ell)), \quad i^\star \in \arg \max_i s_i.$$

Neural MBR instantiates this template with learned reference-based metrics (Freitag et al., 2022), while Universal Self-Consistency uses an LLM-mediated notion of consistency to extend self-consistency beyond exact answer extraction (Chen et al., 2023).

B.4 Proxy misspecification and regularized selection

When a reducer scores candidates using an imperfect proxy j_i , hard argmax selection can overoptimize proxy error as the bank grows (Gao et al., 2023). One stochastic alternative replaces hard argmax selection with

$$\Pr(\hat{y} = Y_i | x, \mathcal{Y}_N) \propto \exp(\alpha j_i), \quad \alpha \geq 0,$$

which is uniform at $\alpha = 0$ and converges as $\alpha \rightarrow \infty$ to uniform selection over the tied maximum-score candidates. More generally, one can combine a proxy score with Bayes-risk regularization (Jinnai et al., 2025), or soften the hard argmax into stochastic selection (Verdun et al., 2025), so that selection is not determined by the single largest proxy score alone. Because the reducer is part of the inference system being evaluated, changing the strength or regularization of selection changes the object under comparison.

B.5 Answer selection versus rationale presentation

When benchmark utility depends only on the final answer, answer selection and rationale presentation are distinct decisions:

$$\hat{a} \in \arg \max_{a \neq \perp} \hat{q}_N(a | x), \quad i^\star \in \arg \max_{i: a_i = \hat{a}} \tilde{T}(x, Y_i).$$

The first rule selects the returned answer. Conditional on that answer, the second uses a rationale-quality score $\tilde{T}$ (for example, the learned score j_i) to choose a sampled rationale for presentation. Both argmax operations use the protocol’s fixed tie rule; the all-invalid case uses its declared fallback. Keeping these decisions separate prevents silent reversion to full-string MAP selection. In neural machine translation, for example, MAP decoding can be a poor output-quality rule (Eikema & Aziz, 2020).

C Prefix-level search: controller and evaluator variants

Prefix search allocates computation using scores on unfinished states. Its evaluator and controller are separate design choices; together, they determine the proposal distribution over leaves. A completed-leaf count also omits computation spent on prefix scoring and tree maintenance.

C.1 Evaluator-controller factorization

A prefix-level search procedure can be written as a pair (S, π_{ctrl}) , where S maps a partial state to a score or summary statistic and π_{ctrl} uses those summaries to choose meta-actions,

$$a_t \sim \pi_{\text{ctrl}}(\cdot \mid x, \mathcal{F}_t, \mathcal{Y}_t, b_t).$$

The action a_t may specify which node to expand, how many successors to generate, how many rollouts to allocate, which states to prune, or when to terminate. Classical MCTS-style procedures (Kocsis & Szepesvári, 2006; Browne et al., 2012), Tree-of-Thoughts-style search (Yao et al., 2023), and RAP (Hao et al., 2023) all instantiate this evaluator-controller pattern with different scorers and control rules; beam and best-first search fit the same factorization. A factorial ablation over S and π_{ctrl} can separate gains due to local evaluation, compute allocation, and their interaction.

C.2 Search-induced proposal shift

Any controller together with its scorer and stopping rule induces an effective leaf distribution

$$q_{\text{search}, B}(y \mid x) = \Pr(\hat{y}_B(x) = y),$$

which generally differs from the decoder-induced proposal $q_\pi(y \mid x)$. When search returns an ordered bank or multiset $\mathcal{Y}$, the corresponding object is its joint law $Q_{\text{search}, B}(\mathcal{Y} \mid x)$ (Section 2.4); per-leaf inclusion probabilities are only marginals of this law. Prefix search therefore changes both compute allocation and the proposal over leaves. The dependence structure and leaf frequencies of a returned bank reflect the controller’s branch-selection decisions. Threshold metrics such as Pass@ k and the profile coordinates $\hat{S}_{k,t}^{\text{bank}}$ (Section 3.4) can still be computed on such banks, but they should be interpreted as descriptive summaries of the search-induced bank unless the experimental protocol explicitly averages over repeated executions of the full search algorithm.

When S is a learned reward or value model, prefix search is susceptible to proxy misspecification in reward-model selection (Gao et al., 2023). Reward-guided tree search may prune or replace low-scoring states (Hung et al., 2025). A branch that would yield a high-utility leaf can therefore be eliminated before that leaf is generated. This failure cannot occur in pure leaf-level reranking after the bank has been generated.

C.3 Compute accounting

Prefix search incurs compute costs for partial-state scoring, rollouts, and tree maintenance, in addition to final generations. Reporting only the number of completed leaves can therefore be misleading: two methods may return the same number of leaves while using very different amounts of model computation and evaluator computation. At minimum, empirical comparisons should separate generated-token cost from evaluator cost in a declared additive work unit, and report latency, throughput, peak memory, and noncommensurate calls separately. Comparisons should also report whether the implementation supports shared-prefix execution. Tree-structured kernels can reuse key-value caches across prefixes, changing the wall-clock frontier for search-based methods (Yao et al., 2025a).

D Reasoning LLMs and Evaluated Models

Open-weight *reasoning models* are trained or configured to emit intermediate steps for multi-step tasks in mathematics, science, and code (Guo et al., 2025; Yang et al., 2025a; Abdin et al., 2025). Their post-training methods use explicit reasoning traces, verifiable rewards, or preference signals, while inference interfaces determine how reasoning is elicited and budgeted. These choices affect reasoning accuracy, output length, and inference cost. Figure 1 presents a selective chronology of the model families.

D.1 Common post-training pipeline archetypes

Open-weight reasoning pipelines recur in three post-training archetypes.

(1) **SFT→RL pipelines.** A common recipe first applies supervised fine-tuning (SFT) on curated chain-of-thought (CoT) demonstrations (Wei et al., 2022), then uses reinforcement learning (RL) to optimize behavior under a reward signal. The reward may reflect verifiable correctness, human or model preferences, or conciseness. Examples include SKY-T1-7B (NovaSky Team, 2025c), LIGHT-R1 (Wen et al., 2025), and Phi-4 reasoning variants (Abdin et al., 2025).

Adjacent designs use SFT-only distillation in the original SKY-T1 (NovaSky Team, 2025b), preference optimization in SKY-T1-FLASH (NovaSky Team, 2025a), or branch-merge distillation in TINYR1 (Sun et al., 2025).

(2) **Distillation-based pipelines.** Distillation uses teacher-generated reasoning traces as SFT data for a smaller student. DeepSeek-R1 supplies traces for its distilled students (Guo et al., 2025), while OpenThoughts3 uses QwQ-32B as its trace generator (OpenThoughts Team, 2025e). Once the traces are fixed, the student is trained with cross-entropy SFT rather than an online RL stage. Workflows such as BESPOKE-STRATOS (Bespoke Labs, 2025a), OPENTHOUGHTS/OPENTHINKER (OpenThoughts, 2025), and OPEN-R1 (Hugging Face, 2025; Open-R1 Team, 2025e) differ in both data volume (from ~10K to > 1M examples) and filtering procedures such as exact-match checks and unit tests.

(3) **Parameter-space fusion and merging.** A third line omits gradient updates during the merge stage and combines aligned pretrained or post-trained checkpoints directly in parameter space. *Select-Calculate-Erase* (SCE), for example, computes a structured weighted merge of aligned checkpoint parameters (Wan et al., 2025). Related FuseAI work includes distribution-level knowledge fusion (Wan et al., 2024a), the fuse-then-merge pipeline of Wan et al. (2025), and SFT+DPO implicit fusion (Yang et al., 2025b); the FUSEO1 family also uses SCE. Branch-merge schemes appear in TINYR1 (Sun et al., 2025). The merge stage requires no new training data, but it is restricted to compatible checkpoints.

D.2 Cross-cutting mechanisms and inference controls

Four mechanism classes distinguish the systems in Figure 1 and Table 2: verified SFT and trace distillation, RL and preference optimization, inference-time control and aggregation, and parameter-space composition.

CoT distillation and verified SFT. CoT distillation transfers teacher-generated solution decompositions to a student through standard SFT. Reasoning pipelines may filter these traces with exact-match checks or unit tests before training (NovaSky Team, 2025b; Bespoke Labs, 2025a; OpenThoughts, 2025). When supervision includes human labels, annotator reliability is a separate source of error (Ghiasvand et al., 2026).

RL and preference optimization. RL-style post-training can optimize policies directly for correctness or preference-aligned behavior. In open-weight work, it appears as outcome-based RL in verifiable domains such as mathematics and code, or as preference optimization that penalizes unnecessarily long reasoning. Direct Preference Optimization (DPO) optimizes a preference objective without fitting the explicit reward model and value function used in Reinforcement Learning from Human Feedback (RLHF) (Rafailov et al., 2023; Ouyang et al., 2022). Reasoning-model objectives may prefer shorter correct chains or penalize redundant verification, as in SKY-T1-FLASH (NovaSky Team, 2025a).

Inference-time scaling and aggregation. Test-time scaling methods allocate inference-time compute by sampling and aggregating multiple candidates, as in majority-vote self-consistency (Wang et al., 2023). Other controls set a “thinking budget” that extends or truncates deliberation, trading token use and latency against accuracy (Muennighoff et al., 2025). These procedures modify the decision rule beyond the base conditional distribution and are therefore part of the evaluated system.

Model merging and fusion. Parameter-space fusion combines checkpoints without gradient updates. FuseO1 checkpoints (FuseAI, 2025a;b) use SCE merging to combine long-CoT and concise-answering models, and the merge itself requires no additional training data. Fusion can change reasoning length and response style while holding the backbone family and parameter count fixed.

D.3 Comparison of representative open-weight reasoning models

Table 2 groups evaluated families and related reference models by post-training mechanism, training-data scale, base model, and inference controls. Because sampling, aggregation, and reasoning budgets can change performance without changing weights, benchmark reports must identify both the checkpoint and the inference protocol.

The mechanisms also impose different dependencies. Verified SFT depends on trace quality and filtering, while RL depends on reward construction and verification. Inference-time controls determine generation and evaluation cost. Parameter-space fusion instead requires compatible checkpoints and can change response length and style without changing the parameter count.

Table 2: Family-level comparison of representative open-weight reasoning models.

Model / Family	Paradigm	Training data	Base model	Technical remarks
DeepSeek-R1 (teacher)	SFT + RL (verifiable)	Multi-stage (cold-start + RL)	DeepSeek-V3-Base	R1-Zero applies RL from a base model; R1 adds cold-start data and multi-stage SFT/RL; teacher for open distillations (Guo et al., 2025).
DeepSeek-R1-Distill [†]	Distillation (supervised fine-tuning)	~800k supervised samples (~600k reasoning)	Qwen2.5 (1.5B–32B), Llama-3.1-8B	Teacher-generated supervised fine-tuning into standard backbones across small and midsize models (Guo et al., 2025; DeepSeek-AI, 2025a).
QwQ-32B	RL reasoning	RL post-training (blog-reported)	Qwen2.5-32B	Dense 32B RL reasoner; reference “thinking” model used as teacher in multiple open pipelines (Qwen Team, 2025e;a).
Qwen3 Thinking [†] (4B / 30B-A3B / Next-80B-A3B)	SFT + RL (thinking)	Long-CoT cold start + reasoning RL + mode fusion + distillation (Yang et al., 2025a)	Qwen3; sparse MoE variants	Thinking-specialized checkpoints; 4B and 30B variants report 262K context; Next-80B uses high-sparsity MoE + hybrid attention with GSPO for RL stability (Qwen Team, 2025c;b;d; Zheng et al., 2025).
s1 / s1.1 (SimpleScaling)	SFT-only (data-minimized) + inference-time control	1k curated, math-dominant CoTs	Qwen2.5-Instruct (multiple sizes)	Minimal SFT with termination control; compatible with sampling and aggregation protocols (Muennighoff et al., 2025).
Sky-T1 [†] (NovaSky)	SFT + preference/RL	~17k curated prompts + preference pairs (Flash)	Qwen2.5-7B/32B	Rejection-sampled and verified traces; Flash uses length-sensitive preference optimization (SimPO-style) + optional rewriting (FCS+1) to reduce overthinking (Li et al., 2025a; NovaSky Team, 2025c;a).
Bespoke-Stratos [†]	Distillation (verified SFT)	17k verified CoTs (math/code/sci)	Students: Qwen2.5-7B/32B / Teacher: DeepSeek-R1	Correctness-filtered distillation (exact-match math, unit tests for code); open Curator pipeline (Bespoke Labs, 2025a).
OpenThinker [†] (OpenThoughts)	SFT-only (curated)	114k–1.2M reasoning traces (math/code/sci/puzzles)	Qwen2.5 (1.5B–32B)	Early releases test answer verification; OpenThoughts3 reports 1,000+ recipe experiments; Evalchemy evaluation (Guha et al., 2026).
Open-R1 [†] (Distill)	SFT-only (verified)	350k verified traces (math/code/sci)	Qwen2.5-Math-7B	Mixture-of-Thoughts: verified DeepSeek-R1 traces; fixed checkpoint for scaling and verifiable-RL follow-ons (Hugging Face, 2025; Open-R1 Team, 2025e).

Table 2: Family-level comparison of representative open-weight reasoning models (continued).

Model / Family	Paradigm	Training data	Base model	Technical remarks
Open-R1 (Math) / OlympicCoder	SFT-only (verified/partial)	220k math (verified); CodeForces-CoTs (~100k traces; partial)	Qwen2.5-Math-7B; Qwen2.5-Coder-7B/32B	Math Verify with a judge fallback for 12% of math samples; code traces use public tests and are not exhaustively verified; decontamination is documented (Open-R1 Team, 2025f;a; Penedo et al., 2025).
LIMO / LIMO-v2 / LIMR [†] (GAIR)	SFT-only (data-minimized) + RL (verifiable)	800 curated math; 1.4k RL-selected items	Qwen2.5-32B-Instruct (LIMO); Qwen2.5-Math-7B (LIMR)	Small-data SFT and learning-impact selection for RL; v2 pairs an updated 800-example dataset with a 32B checkpoint (Ye et al., 2025; GAIR, 2025b;a; Li et al., 2025b).
Light-R1 / TinyR1 [†] (Qihoo360)	SFT + preference/RL; merge (branch-merge)	76k + 3k CoTs; verified preference pairs	Qwen2.5; DeepSeek-R1-Distill backbones	Curriculum/step-wise SFT + DPO; GRPO on distilled variants; TinyR1 uses branch specialization followed by merging across domains (Wen et al., 2025; Sun et al., 2025).
FuseO1 [†] (FuseAI)	Merge (parameter fusion)	N/A	DeepSeek-R1-Distill-32B, QwQ-32B-Preview, Sky-T1-32B (incl. Flash)	SCE-style parameter-space merging of long-CoT and concise models; requires no new data during merging (FuseAI, 2025c).
gpt-oss-20b [†] (OpenAI)	CoT RL (MoE)	CoT RL post-training (model card)	OpenAI MoE (21B total / 3.6B active)	MXFP4 quantization; low, medium, and high reasoning-effort settings vary token use, latency, and accuracy (OpenAI et al., 2025).
Phi-4 Reasoning [†] (Microsoft)	SFT + RL (verifiable)	Distilled traces + ~6k RL math	Phi-4 (14B)	Teachable-prompt selection + long-CoT SFT; outcome-based RL in “plus”; evaluated with inference-time budget and sampling controls (Abdin et al., 2025).
Nemotron Reasoning [†] (NVIDIA)	SFT-only (OpenReasoning); SFT + verifiable RL (AceReason)	Large-scale synthetic post-training	Qwen2.5 (OpenReasoning, AceReason); hybrid Mamba-Transformer (Nano v2)	OpenReasoning is SFT-only; AceReason uses SFT followed by verifiable RL; Nano v2 uses a hybrid Mamba-Transformer architecture (NVIDIA, 2025; Liu et al., 2025d; NVIDIA et al., 2025).
EXAONE-4.0 [†] (LG AI)	SFT + preference/RL	Multi-domain math/code/science mixtures	EXAONE-4.0-32B / 1.2B	Unified modes (chat vs. reasoning); long-context support; preference/RL stages to balance correctness and verbosity (Bae et al., 2025).

[†] At least one checkpoint or configuration from this family appears in the 20-configuration repeated-mathematics roster in Section G.4. Unmarked rows provide broader field context for that roster.

D.3.1 DeepSeek-R1 and distilled students: RL post-training and supervised distillation

DeepSeek-R1 uses large-scale RL on tasks with verifiable outcomes. **DeepSeek-R1-Zero** is trained with RL directly from a base model, without an SFT warm start. The report describes self-verification and reflection in its outputs, along with lower readability and language mixing. **DeepSeek-R1** adds cold-start data, SFT, RL, and rejection sampling to address those output problems while retaining the reasoning behavior (Guo et al., 2025).

The **DeepSeek-R1-Distill** family transfers R1 outputs into standard dense backbones rather than running RL directly on smaller policies. The released students include Qwen2.5-based checkpoints at 1.5B, 7B, 14B, and 32B, and a Llama-3.1-based 8B checkpoint (Guo et al., 2025). The Hugging Face model cards report

that the Qwen2.5 students are fine-tuned on $\sim 800\text{K}$ DeepSeek-R1-curated samples (DeepSeek-AI, 2025a). The project reports that these distilled checkpoints outperform its attempts to train smaller models directly with RL (DeepSeek-AI, 2025b).

D.3.2 Qwen reasoning models: thinking-mode post-training and sparse MoE

Qwen treats “thinking” as an explicit post-training target and uses both dense and sparse-MoE architectures. **QwQ-32B** is a dense 32B model post-trained with RL; Qwen reports benchmark results comparable to DeepSeek-R1 and o1-mini (Qwen Team, 2025e;a). Unlike the distilled families above, QwQ-32B is post-trained directly for reasoning.

Qwen3 separates *thinking* and *non-thinking* behavior through a staged pipeline: long-CoT cold start, reasoning RL, “thinking mode fusion,” general RL, and strong-to-weak distillation (Yang et al., 2025a). The evaluated **Qwen3-4B-Thinking-2507** checkpoint has no mode toggle and a reported 262K context window (Qwen Team, 2025c). **Qwen3-30B-A3B-Thinking-2507** uses sparse MoE, with 30.5B total and 3.3B active parameters (128 experts, 8 active), and reports the same context length (Qwen Team, 2025b).

Outside the evaluated roster, **Qwen3-Next-80B-A3B-Thinking** combines hybrid attention with a higher-sparsity MoE. Its model card attributes RL stability and efficiency in this setting to Group Sequence Policy Optimization (GSPO) (Qwen Team, 2025d; Zheng et al., 2025). The checkpoint’s default prompt template enforces thinking mode.

D.3.3 NovaSky: verified trace distillation and length control

The Sky-T1 line uses small verified corpora to train long-CoT behavior in instruction-tuned backbones; later variants add preference optimization or RL (NovaSky Team, 2025b;a;c).

Sky-T1-32B-Preview fine-tunes QWEN2.5-32B-INSTRUCT on roughly 17K demonstrations distilled from QwQ-32B-PREVIEW and selected by rejection sampling. The reported math and code results improve over the base model. In the paper’s perturbation experiments, shuffling or deleting reasoning steps causes larger accuracy losses than perturbing surface tokens (Li et al., 2025a; NovaSky Team, 2025b).

Sky-T1-32B-Flash (NovaSky Team, 2025a) penalizes overthinking during training. It forms preference pairs from the shortest and longest correct candidates and adds hard-negative pairs that contrast a short incorrect answer with a long correct one on difficult prompts. The method then applies a length-sensitive, SimPO-style preference objective (Meng et al., 2024). An optional “FCS+1” rewrite retains the first correct mathematics solution and one additional solution; code outputs are not rewritten.

Sky-T1-7B (NovaSky Team, 2025c) uses an iterative SFT/RL schedule starting from QWEN2.5-MATH-7B. It first applies SFT on a small verified set of QwQ-distilled traces, followed by RL with PRIME-style prompt filtering and rollouts on Eurys-2-RL-Data (Cui et al., 2026). A second distillation and SFT stage precedes the final RLOO-based RL stage (Ahmadian et al., 2024).

D.3.4 Bespoke-Stratos: reasoning distillation as data curation

Bespoke-Stratos uses verified teacher traces as cross-entropy SFT data. The pipeline distills DEEPSEEK-R1 outputs into BESPOKE-STRATOS-17K and fine-tunes QWEN2.5-INSTRUCT backbones at multiple scales (Bespoke Labs, 2025a).

Bespoke-Stratos-17k contains 16.7K examples produced with a Sky-T1-style distillation recipe (Bespoke Labs, 2025b). Candidate traces from DEEPSEEK-R1 are filtered by strict answer checks for mathematics and unit-test execution for code before SFT.

The project releases two checkpoints. **Bespoke-Stratos-32B** fine-tunes QWEN2.5-32B-INSTRUCT on this dataset, and **Bespoke-Stratos-7B** applies the same procedure to QWEN2.5-7B-INSTRUCT (Bespoke Labs, 2025c;d). The project reports improvements over both base models on its mathematics and code evaluations (Bespoke Labs, 2025a).

D.3.5 OpenThoughts/OpenThinker: scaling supervised reasoning data

OpenThoughts studies SFT-only reasoning post-training with composed synthetic traces, testing how performance changes with the volume and composition of the supervision data (Guha et al., 2026; OpenThoughts, 2025).

The first release, **OpenThoughts-114K** (OpenThoughts Team, 2025c), generates long-form traces for mathematics, science, code, and puzzle prompts and verifies correctness before assembling the dataset. The corresponding **OpenThinker** models use cross-entropy SFT on QWEN2.5 instruction-tuned backbones and are evaluated with EVALCHEMY. This shared training and evaluation code supports comparisons between verified and unverified trace sets. Later ablations report that answer verification helps at 32B but hurts at 7B. OpenThoughts3 omits answer verification (Guha et al., 2026).

OpenThoughts2-1M contains one million curated examples and is used to train **OpenThinker2-32B**; the authors report matching DEEPSEEK-R1-DISTILL-32B on AIME and LiveCodeBench (OpenThoughts Team, 2025d;a). The **OpenThoughts3-1.2M** recipe was selected through more than 1,000 controlled experiments and uses QWQ-32B as its trace teacher (OpenThoughts Team, 2025e). **OpenThinker3-7B** is trained by SFT on the 1.2M-example mixture, reported as 850K mathematics, 250K code, and 100K science examples, and its model card reports results on AIME’25, LiveCodeBench, and GPQA Diamond (OpenThoughts Team, 2025b;e). The evaluation includes **OpenThinker2-32B** and **OpenThinker3-1.5B**, trained with the OpenThoughts2 and OpenThoughts3 recipes, respectively.

D.3.6 OpenAI gpt-oss-20b

gpt-oss-20b is an MoE transformer post-trained with chain-of-thought RL (OpenAI, 2025b; OpenAI et al., 2025). It uses MXFP4 quantization and has approximately 21B total parameters, with 3.6B active per token. Its inference interface exposes three reasoning-effort settings: `low`, `medium`, and `high` (OpenAI, 2025a).

D.3.7 Microsoft Phi-4 reasoning models

Microsoft’s Phi-4 reasoning line specializes a 14B foundation model through curated supervision and, for one variant, a short outcome-based RL stage. Starting from PHI-4 (Abdin et al., 2024), **Phi-4-reasoning** applies SFT to prompts described as “teachable” (diverse and challenging but within the student’s learnable regime), paired with long-form demonstrations from O3-MINI. The report studies this prompt-selection criterion and evaluates longer reasoning budgets and repeated sampling as inference controls (Abdin et al., 2025).

Phi-4-reasoning-plus adds a short outcome-based RL stage that rewards final-answer correctness on a verified mathematics set. The report associates this stage with higher benchmark accuracy and longer average reasoning traces (Abdin et al., 2025). Microsoft also releases compact variants. **Phi-4-mini-reasoning** (3.8B) uses mid-training on distilled long-CoT data, long-CoT SFT, rollout-DPO preference optimization, and verifiable-reward RL for mathematics (Xu et al., 2025). **Phi-4-mini-flash-reasoning** uses a hybrid SAMBAY design and Differential Attention for 64K-context reasoning (Microsoft, 2025). For a 2K-token prompt and 32K-token generation under vLLM, the accompanying work reports up to 10× the decoding throughput of Phi-4-mini-reasoning. It also reports higher scores on MATH-500, AIME 2024, AIME 2025, and GPQA Diamond, although the Flash variant does not use RL post-training (Ren et al., 2025).

D.3.8 NVIDIA Nemotron reasoning models

The evaluated NVIDIA releases include **OpenReasoning-Nemotron-1.5B**, a dense Qwen2.5-derived SFT model for mathematics, code, and science (NVIDIA, 2025), and **AceReason-Nemotron-1.1-7B**, which uses an SFT-and-RL pipeline for mathematics and code (Liu et al., 2025d). Related NVIDIA work releases the OPENMATHREASONING (Moshkov et al., 2025) and OPENCODEREASONING (Ahmad et al., 2025a;b) data and model pipelines.

NVIDIA-Nemotron-Nano-9B-v2 uses a hybrid Mamba–Transformer architecture with controllable reasoning (NVIDIA et al., 2025). The roster therefore contains a 1.5B dense baseline, a 7B SFT-and-RL specialist, and a 9B hybrid model with reasoning controls.

D.3.9 Open-R1 and OlympicCoder: open reproductions for math and competitive programming

Open-R1 (Hugging Face, 2025) publishes a staged reproduction of DEEPSEEK-R1-style post-training, including its data mixtures, verification procedures, and inference protocols.

A central component is **Mixture-of-Thoughts** (Open-R1 Team, 2025b), a $\sim 350\text{K}$ dataset of verified traces distilled from DEEPSEEK-R1 across mathematics, code, and science. **OpenR1-Distill-7B** is trained by SFT on this mixture and provides a fixed checkpoint for analyzing repeated sampling, aggregation, and subsequent verifiable-reward RL (Open-R1 Team, 2025e). Open-R1 also provides a mathematics-only corpus, **OpenR1-Math-220k**, derived from NuminaMath 1.5 with multiple traces per problem and automatic verification by MATH VERIFY. When rule-based checks are insufficient, LLAMA-3.3-70B-INSTRUCT is the fallback judge; the dataset card reports that this applies to 12% of samples (Ben Allal et al., 2025; Open-R1 Team, 2025f).

OlympicCoder applies the same approach to competitive programming, where verification depends on test coverage. **CodeForces-CoTs** contains more than 10K CodeForces problems and nearly 100K C++/Python traces generated by DEEPSEEK-R1. The dataset is not exhaustively filtered; about 84% of its Python solutions pass the public tests (Open-R1 Team, 2025a). **OlympicCoder-7B** and **OlympicCoder-32B** fine-tune QWEN2.5-CODER instruction models on decontaminated CodeForces-style CoT planning data and are evaluated on an IOI’2024 subset, LiveCodeBench, and other standardized competitive-programming benchmarks (Open-R1 Team, 2025d;c; Penedo et al., 2025). The mathematics corpus uses automatic verification with an LLM fallback judge, whereas the code corpus relies on public tests and is only partially verified.

D.3.10 GAIR “Less is More”: data-minimized reasoning and impact-aware RL

GAIR’s “Less is More” line tests whether data selection can substitute for training-set volume. For **LIMO**, candidate solutions to 2,125 mathematics problems are sampled from DEEPSEEK-R1, DEEPSEEK-R1-DISTILL-QWEN-32B, and QWQ-32B. The highest-scoring solution for each problem is retained, the resulting problem–solution pairs are ranked, and the top 800 are used to fine-tune QWEN2.5-32B-INSTRUCT (Ye et al., 2025). The study evaluates the resulting checkpoint with additional inference-time compute, treating data selection and inference allocation as separate choices. **LIMO-v2** pairs an updated 800-example dataset with a corresponding QWEN2.5-32B-INSTRUCT checkpoint release (GAIR, 2025b;a).

LIMR introduces *Learning Impact Measurement* (LIM), which selects RL prompts by matching reward trajectories to the model’s learning dynamics. In its experiments, RL on 1,389 selected prompts matches or exceeds RL on the full 8,523-prompt pool. The reported comparison also differs by model scale: the smallest SFT sets work best at larger scales, whereas LIM-selected RL improves the smaller models tested (Li et al., 2025b).

D.3.11 FuseAI/FuseO1: parameter-space merging

Checkpoint merging combines trained models directly in parameter space without gradient updates. FuseAI studies heterogeneous model fusion (Wan et al., 2025).

FuseAI work includes distribution-level knowledge fusion (Wan et al., 2024a), FuseChat’s fuse-then-merge pipeline (Wan et al., 2025), and SFT+DPO implicit fusion (Yang et al., 2025b). FuseChat introduces **SCE** (Select–Calculate–Erase), a parameter-matrix merge rule built around fusion vectors (weight differences from a pivot) and a structured per-matrix procedure (Wan et al., 2025). The **FuseO1** family combines DEEPSEEK-R1-DISTILL-QWEN-32B, QWQ-32B-PREVIEW, and SKY-T1-32B variants through SCE, without gradient updates or new data during the merge (FuseAI, 2025c;a;b). Including a Flash parent is intended to combine long- and short-reasoning behavior.

D.3.12 Light-R1 and TinyR1: curriculum post-training and branch-merge distillation

Light-R1 uses curriculum-style post-training to induce long-CoT behavior; TinyR1 trains domain-specific branches and then merges them.

Light-R1 uses two-stage curriculum SFT followed by semi-on-policy DPO to train long-form reasoning and response style (Wen et al., 2025). In its “from scratch” setting, **Light-R1-32B** starts from QWEN2.5-32B-INSTRUCT and uses mathematics-focused data (Qihoo360, 2025b). A 3K “stage-2” hard long-CoT set is then applied to DEEPSEEK-R1-DISTILL-QWEN checkpoints, producing **Light-R1-7B-DS** and **Light-R1-32B-DS** (Qihoo360, 2025d;c). The report attributes higher benchmark scores for these students to the additional SFT stage. GRPO post-training produces **Light-R1-14B-DS**; the report observes higher reward alongside longer outputs (Wen et al., 2025; Qihoo360, 2025a).

TinyR1-32B-Preview uses Branch-Merge Distillation. It distills DEEPSEEK-R1 into domain-specific branches through SFT and then merges them. Relative to DEEPSEEK-R1-DISTILL-QWEN-32B, the paper reports average gains of 5.5 points in mathematics, 4.4 in code, and 2.9 in science. TinyR1 also approaches DEEPSEEK-R1 on AIME’24 under the reported evaluation settings (Sun et al., 2025).

D.3.13 SimpleScaling s1.1: small-data SFT and budget forcing

SimpleScaling’s s1.1 family uses SFT on a small curated reasoning set and budget forcing at inference time, replacing the earlier Gemini-generated traces with DeepSeek-R1 traces. Budget forcing truncates or extends a reasoning trace to control test-time compute (Muennighoff et al., 2025).

Across several Qwen model sizes, s1.1 combines sequential budget forcing with parallel methods such as majority voting and REBASE; s1.1-32B is the primary reference model (Muennighoff et al., 2025).

D.3.14 LG EXAONE-4.0: unified chat and reasoning modes

EXAONE-4.0 treats chat-style assistance and explicit reasoning as two modes of one model family. Its unified post-training recipe jointly trains non-reasoning and reasoning behavior, followed by preference-learning stages for correctness, brevity, and language consistency in the 32B and 1.2B models (Bae et al., 2025).

The 32B model uses a 3:1 ratio of sliding-window to global-attention layers, removes rotary positional embeddings from global-attention layers, and applies QK-Reorder-Norm. Long-context post-training accompanies these architectural changes. Post-training includes an RL stage and multi-turn, long-horizon tool-use examples with execution feedback (Bae et al., 2025).

E Prompt and Inference Interfaces

Configurations use their model-native reasoning interfaces. Most receive a zero-shot mathematics instruction requesting step-by-step reasoning and a boxed final answer; gpt-oss also uses low, medium, and high effort controls. Scoring uses the extracted final answer. In the repeated-mathematics block, gpt-oss prompt coverage varies by task, as detailed in Section G.4.

F Aggregation Reference Interface

The reference interface implements the aggregation protocol in Section 3.3 through three stages: evidence reducers map confidence or reward signals to candidate scores; fixed-bank rules select an answer from aligned answer and score arrays; and online predicates decide whether to stop sampling or generation from the history observed so far. Separating these stages allows the same candidate bank to be evaluated under several decision rules and the same rule to be paired with different evidence sources, without conflating candidate generation with post-generation selection.

The interface accepts a one-dimensional candidate bank for one problem or a two-dimensional array for a batch. Score-free consensus, score-based selection, and across-rollout stopping use the same extracted-answer

representation. Fixed-bank routines may also return the index of a representative candidate when the selected answer needs a rationale. In Listing 1, plurality selects “A”, whereas Best-of- N and softmax-weighted voting select “B”; the causal stopping predicate reads only the revealed answer prefix.

```

1  answers = ["A", "A", "A", "B"]
2  scores = [0.30, 0.30, 0.30, 0.85]
3
4  plurality = agg.majority_vote(answers)           # "A"
5  best = agg.best_of_n(answers, scores)           # "B"
6  weighted = agg.softmax_weighted_vote(
7      answers, scores, temperature=0.1
8  )                                               # "B"
9
10 observed = ["A"] * 8 + ["B"] * 2
11 stop, probability = agg.adaptive_consistency_stop(
12     observed, threshold=0.95, return_prob=True
13 )                                               # (True, 0.9673...)

```

Listing 1: Reference-interface pseudocode for fixed-bank aggregation and causal stopping. Higher candidate scores indicate stronger evidence.

G Data, Experiments, and Reproducibility

We release 1,403,520 attempts across repeated mathematics, signal-rich mathematics, and field-balanced SuperGPQA, with 80 attempts per model-question pair. Table 3 shows that the 3,600-question field-balanced SuperGPQA collection accounts for most released attempts, whereas the two mathematics collections cover 306 questions across nine tasks, so the aggregate release count combines collections with very different question and task coverage. A *response* is one model completion, and a *bank* comprises the responses from one model configuration on a fixed question set.

Collection	Model cfgs.	Tasks	Questions	Attempts
Repeated mathematics	20	4	120	192,000
Signal-rich mathematics	4	5	186	59,520
Field-balanced SuperGPQA	4	1	3,600	1,152,000
Total	—	—	—	1,403,520

Table 3: Coverage of the public response collections.

G.1 Data sources

Across the benchmark sources in Table 4, question-set scale ranges from 30-question competition sets to 12,032 MMLU-Pro questions; the block-level estimates therefore rest on markedly different numbers of questions.

Block	Source	Questions
Broad	MMLU-Pro (TIGER-Lab, 2024; Wang et al., 2024)	12,032
Broad	BBH (SaylorTwift, 2024; Suzgun et al., 2023)	6,511
Repeated math	AIME’24 (Jia, 2024)	30
Repeated math	AIME’25 (Zhang & Math-AI, 2025)	30
Repeated math	HMMT’25 (MathArena, 2025c)	30
Repeated math	BrUMO’25 (MathArena, 2025a)	30
Signal-rich math	Five MathArena-hosted sets	186
Broad extension	SuperGPQA (M-A-P Team, 2025; M-A-P et al., 2025)	3,600

Table 4: Question sources and sizes.

The signal-rich study combines all 186 questions from AIME’26, HMMT Feb.’26, HMMT Nov.’25, CMIMC’25, and SMT’25 (MathArena, 2026a;b; 2025d;b;e). Its results use our evaluation protocol.

G.2 Metrics, reducers, and uncertainty

For a question with N responses, let c be the number marked correct by the benchmark adapter. The exact without-replacement coordinates used throughout are

$$\text{Pass@ } k = 1 - \frac{\binom{N-c}{k}}{\binom{N}{k}}, \quad \text{pass}^k = \frac{\binom{c}{k}}{\binom{N}{k}}. \quad (9)$$

The first asks whether a size- k subset contains at least one correct candidate; the second asks whether every candidate in that subset is correct. They describe a bank and do not specify which response a system submits.

We evaluate submitted-answer accuracy by replaying reducers on subsets of the same bank. Budgets are $k \in \{1, 2, 4, 8, 16, 32, 64, 80\}$. A counter-derived seed gives each question a reproducible random permutation, and prefixes form nested uniform subsets without replacement. In the 20-configuration study we enumerate all 80 singleton choices and use 200 paired subset replays for each $k > 1$. The signal-rich mathematics study uses 2,000 replays for $1 < k < 80$, an exact average over all singletons at $k = 1$, and the full bank at $k = 80$. Ties are resolved by the earliest candidate in the seeded random order. A reducer’s $k = 80$ value is therefore exact conditional on that fixed tie-breaking order, not an average over every possible permutation.

The reported intervals resample questions after integrating over subset replays. Experiment 2 uses 2,000 percentile bootstrap replicates; the signal-rich mathematics analysis uses 10,000. These intervals are conditional on the observed 80-response banks. They measure sensitivity to the question composition, not the additional variability that would arise from generating a new bank. Pooled summaries are item-weighted micro-averages. For SuperGPQA, field-balanced summaries give each of the 72 constructed fields equal weight because every field contributes 50 questions.

The available signals constrain which analyses a bank supports: Table 5 maps each diagnostic or reducer to its required inputs and distinguishes reference-free reducers from Compass Best-of- N , which requires a benchmark reference and is used only diagnostically.

Reducer or statistic	Required signals	Interpretation
Pass@ k , pass k	Binary outcome	Exact bank diagnostic
String plurality	Extracted answer	Deployable terminal reducer; literal strings
Sequence/mean log probability	Chosen-token log probabilities	Deployable confidence heuristic
Normalized-likelihood vote	Answer string and response likelihood	Deployable weighted terminal vote
Pointwise Best-of- N	Reference-free Qwen score	Deployable after paying verifier cost
Compass Best-of- N	Question, reference, and candidate	Reference-assisted diagnostic only

Table 5: Inputs and roles of the reported analyses. “Deployable” here means that the reducer does not require a benchmark reference answer; its compute is still additional to candidate generation.

G.3 Broad knowledge and symbolic reasoning

Protocol. The broad block evaluates 27 model configurations on all 12,032 MMLU-Pro and 6,511 BBH questions (Wang et al., 2024; Suzgun et al., 2023). MMLU-Pro uses final-option exact match, whereas BBH uses its flexible extractor.

Generation is zero-shot, with temperature 0.6, top- $p = 0.95$, frequency penalty 0.1, a 32,768-token cap, and a fixed seed.

Response counts. The 27 configurations produce 500,661 responses across the two suites. This block lacks token-level probabilities and structured finish reasons, so it supports neither likelihood-based reducers nor

reliable termination analysis. The configuration orderings in Table 6 differ between MMLU-Pro and BBH, so the suite-specific exact-match columns are retained as separate descriptive results rather than pooled into one ranking.

Configuration	MMLU-Pro	BBH
Bespoke-Stratos-32B	67.37	75.41
Bespoke-Stratos-7B	46.68	51.50
DeepSeek-R1-Distill-Llama-8B	29.42	64.57
DeepSeek-R1-Distill-Qwen-1.5B	3.13	25.97
DeepSeek-R1-Distill-Qwen-14B	48.85	77.88
DeepSeek-R1-Distill-Qwen-32B	46.02	81.68
DeepSeek-R1-Distill-Qwen-7B	6.68	53.11
FuseO1-DeepSeekR1-QwQ-32B-Preview	45.05	81.22
FuseO1-DeepSeekR1-QwQ-SkyT1-32B-Preview	44.96	82.20
FuseO1-DeepSeekR1-QwQ-SkyT1-Flash-32B-Preview	44.52	81.45
FuseO1-DeepSeekR1-Qwen2.5-Instruct-32B-Preview	46.62	82.14
LIMO	65.61	74.41
Light-R1-14B-DS	38.23	74.09
Light-R1-32B	55.60	74.44
Light-R1-7B-DS	7.68	54.57
OlympicCoder-32B	50.78	57.79
OlympicCoder-7B	40.88	56.89
OpenR1-Qwen-7B	15.97	31.45
OpenThinker-32B	66.62	75.26
OpenThinker-7B	47.45	52.22
QwQ-32B	70.47	58.45
Qwen2.5-32B-Instruct	67.10	56.52
Sky-T1-32B-Flash	60.29	79.14
Sky-T1-32B-Preview	59.48	79.67
Sky-T1-7B	15.08	39.35
TinyR1-32B-Preview	58.32	77.39
s1.1-32B	64.23	41.48

Table 6: Broad-block exact match in percent. BBH uses its flexible extractor. The columns are separate descriptive results and are not averaged.

Empty BBH extractions remain in the denominator and are scored incorrect.

G.4 Twenty configurations on 2024–2025 mathematics

Grid and models. The four sets are AIME’24, AIME’25, HMMT’25, and BrUMO’25, with 30 questions each. HMMT’25 and BrUMO’25 are distributed through MathArena, but the four-set combination is not itself a MathArena benchmark. Twenty configurations generate 80 responses for each of 120 questions, yielding 192,000 responses.

Generation. Generation uses temperature 0.6, top- $p = 0.95$, one completion, and a 32,768-token cap. Realized-token log probabilities and ranks are available at every generation position, but full next-token distributions are not. These signals support sequence-likelihood, mean-log-probability, and realized-rank summaries, but not entropy or other full-distribution methods. All non-gpt-oss configurations use the common boxed-answer instruction. Within the three gpt-oss effort configurations, that instruction is present for 80% of the AIME’24 and AIME’25 attempts and absent from the BrUMO’25 and HMMT’25 attempts. Comparisons involving these configurations therefore combine model, task, and prompt-interface differences.

CompassVerifier-7B. CompassVerifier-7B assigns an outcome score whose A/B/C labels mean correct, incorrect, and invalid (Liu et al., 2025c); we use the probability assigned to A. Scores or parsed answers that

Configuration	Acc.@1	Pass@8	Pass@80
Qwen3-30B-A3B-Thinking-2507	75.56	85.83	91.67
Qwen3-4B-Thinking-2507	66.55	80.40	87.50
Phi-4-reasoning-plus	65.56	84.19	91.67
gpt-oss-20b high	63.99	83.94	90.83
gpt-oss-20b medium	63.65	86.49	93.33
AceReason-Nemotron-1.1-7B	60.74	74.70	80.83
Phi-4-reasoning	59.97	82.81	90.83
OpenThinker2-32B	59.75	76.77	82.50
FuseO1-DeepSeekR1-QwQ-SkyT1-Flash-32B-Preview	58.52	74.05	81.67
Light-R1-14B-DS	58.27	74.09	81.67
NVIDIA-Nemotron-Nano-9B-v2	54.72	72.12	79.17
LIMO-v2	53.45	73.33	82.50
gpt-oss-20b low	47.39	72.39	85.00
EXAONE-4.0-1.2B	44.29	67.78	83.33
OpenR1-Distill-7B	43.68	65.66	78.33
OpenThinker3-1.5B	43.38	62.13	75.00
OpenReasoning-Nemotron-1.5B	40.73	62.07	78.33
DeepSeek-R1-Distill-Qwen-1.5B	25.30	47.47	64.17
Sky-T1-32B-Flash	25.27	43.95	55.83
Bespoke-Stratos-7B	18.39	36.22	54.17

Table 7: Complete fixed-roster results for the 2024–2025 mathematics block, in percent. Acc.@1 averages the 80 response outcomes per question; Pass@ k is the exact without-replacement discovery statistic.

are unavailable are excluded without imputation. Because the verifier sees the gold answer, we use it only as a reference-assisted diagnostic.

Across the complete roster in Table 7, Pass@80 exceeds Acc.@1, the mean single-response accuracy, for every configuration; the full-bank discovery statistic nevertheless does not define submitted-answer accuracy.

Qwen3-30B-A3B-Thinking-2507	$k = 1$	$k = 8$	$k = 80$
Pass@ k	75.56	85.83	91.67
pass ^k	75.56	61.50	43.33
String plurality	75.56	77.74	78.33
Sequence-log-probability Best-of- N	75.56	78.45	77.50
Mean-log-probability Best-of- N	75.56	74.56	65.83
Compass Best-of- N (reference-assisted)	75.51	84.23	89.17

Table 8: Selected repeated-mathematics results in percent, pooled over 120 questions. The $k = 1$ reducer values enumerate all singletons; $k = 8$ uses 200 paired subset replays; $k = 80$ is the full bank. Negative perplexity is identical to mean-log-probability selection.

For Qwen3-30B-A3B-Thinking-2507, Table 8 shows an availability–selection gap at $k = 80$: Pass@80 is 91.67%, whereas the best deployable terminal reducer shown, string plurality, reaches 78.33%. The leading configuration’s Acc.@1 has a 95% prompt-bootstrap interval of [68.81, 82.18]. Across the 20 configurations, the mean Spearman correlation between subset and full-bank accuracy rankings is 0.967 at $k = 1$ and 0.992 at $k = 8$. This stability describes the present roster; it is not a general sample-size guarantee.

G.5 Signal-rich competition mathematics

Generators and prompts. The generators are Qwen3.6-35B-A3B and gpt-oss-20b (Qwen Team, 2026; OpenAI, 2025b). Each receives the problem in a zero-shot prompt that requests step-by-step reasoning and a boxed final answer; no reference or evaluator signal is shown. Boxed answers are checked for numeric or symbolic equivalence (ModelScope Team, 2024).

Qwen uses temperature 1, top- $p = 0.95$, top- $k = 20$, and presence penalty 1.5. GPT-OSS uses temperature 1, top- $p = 1$, no top- k truncation, and no presence penalty. Both use one completion and an 81,920-token generation cap.

Effective effort and sampling. Each model-effort condition contains $186 \times 80 = 14,880$ responses. The four analyzed banks are Qwen and gpt-oss at low, medium, and high effort. Three additional gpt-oss runs used medium effort and serve only as diagnostics; they are not pooled with the analyzed medium bank. Their Acc.@1 values range from 63.08% to 63.47%, compared with 62.80% for the analyzed medium bank.

Token-level signals. We observe realized-token log probabilities and ranks and at most 20 alternatives per completion step. Ranks and probability mass beyond the first 20 alternatives are unavailable, and the recorded probabilities precede decoding penalties and truncation.

G.6 Verifier implementations and diagnostics

CompassVerifier scores final-answer outcomes with access to the reference answer. The pointwise Qwen evaluator instead scores problem understanding, reasoning validity, and conclusion support without a reference; its score is used for reference-free Best-of- N (Table 9).

	CompassVerifier-3B (OpenCompass)	Pointwise Qwen adaptation
Input	Problem, reference answer, candidate response	Problem and one candidate response; no reference or other candidate
Judgment	A/B/C final-answer outcome; reasoning explicitly ignored	Problem understanding, reasoning validity, and conclusion support
Score	Direct A/B/C softmax; null-adjusted contextual A/B/C softmax	Expected A-T ordinal score per criterion, then mean
Role	Reference-assisted outcome diagnostic	Reference-free Best-of- N evidence
Main caveat	Gold access and neither distribution is calibrated	Ordinal, not calibrated; Qwen evaluates its own Qwen traces

Table 9: The two verifier pipelines. The pointwise pipeline is inspired by LLM-as-a-Verifier (Kwok et al., 2026) but is not its pairwise tournament. CompassVerifier follows the released outcome-verifier prompt and scoring formulation (Liu et al., 2025c).

CompassVerifier-3B (OpenCompass). The official prompt supplies the problem, standard answer, and candidate. It asks for A (correct), B (incorrect), or C (invalid), instructs the verifier to compare final answers, and explicitly says to ignore errors in the reasoning when the final answer is correct (Liu et al., 2025c). From the label log probabilities (ℓ_A, ℓ_B, ℓ_C) , the direct score is

$$p_{\text{direct}}(z) = \text{softmax}_{z \in \{A, B, C\}}(\ell_z).$$

The contextual variant subtracts a null-prompt baseline before normalizing; let ℓ_z^{slot} and ℓ_z^{null} denote the label log probabilities under the populated and null prompts:

$$p_{\text{ctx}}(z) = \text{softmax}_{z \in \{A, B, C\}} \left(\frac{\ell_z^{\text{slot}} - \ell_z^{\text{null}}}{1.5} \right).$$

This adjustment is a diagnostic used by our pipeline; it is not a calibrated correctness probability or the production label of CompassVerifier.

For inputs exceeding the context window, only the middle of the candidate is shortened, preserving its prefix and suffix as well as the question and reference.

Reference-free pointwise Qwen evaluator. The corpus-wide score adapts the expected-score idea of LLM-as-a-Verifier to one candidate at a time (Kwok et al., 2026). Qwen critiques problem understanding, reasoning validity, and conclusion support, then assigns an A-T ordinal distribution to each criterion. Unlike

the cited pairwise method, this evaluator ranks candidates directly without a tournament or Bradley–Terry aggregation.

Let $q_j(L)$ be the resulting distribution for criterion j , with $v(A) = 20, \dots, v(T) = 1$. We compute

$$r_j = \frac{\sum_{L=A}^T q_j(L)v(L) - 1}{19}, \quad s_{\text{point}} = \frac{1}{3} \sum_{j=1}^3 r_j.$$

The score is in $[0, 1]$, but the construction is ordinal rather than probabilistic calibration, and low-mass letters may be censored by the returned token probabilities. Qwen judging Qwen-generated responses is self-evaluation; judging gpt-oss responses is cross-model evaluation. The reported selection curves and intervals condition on the observed verifier scores and do not include evaluator-rerun variability.

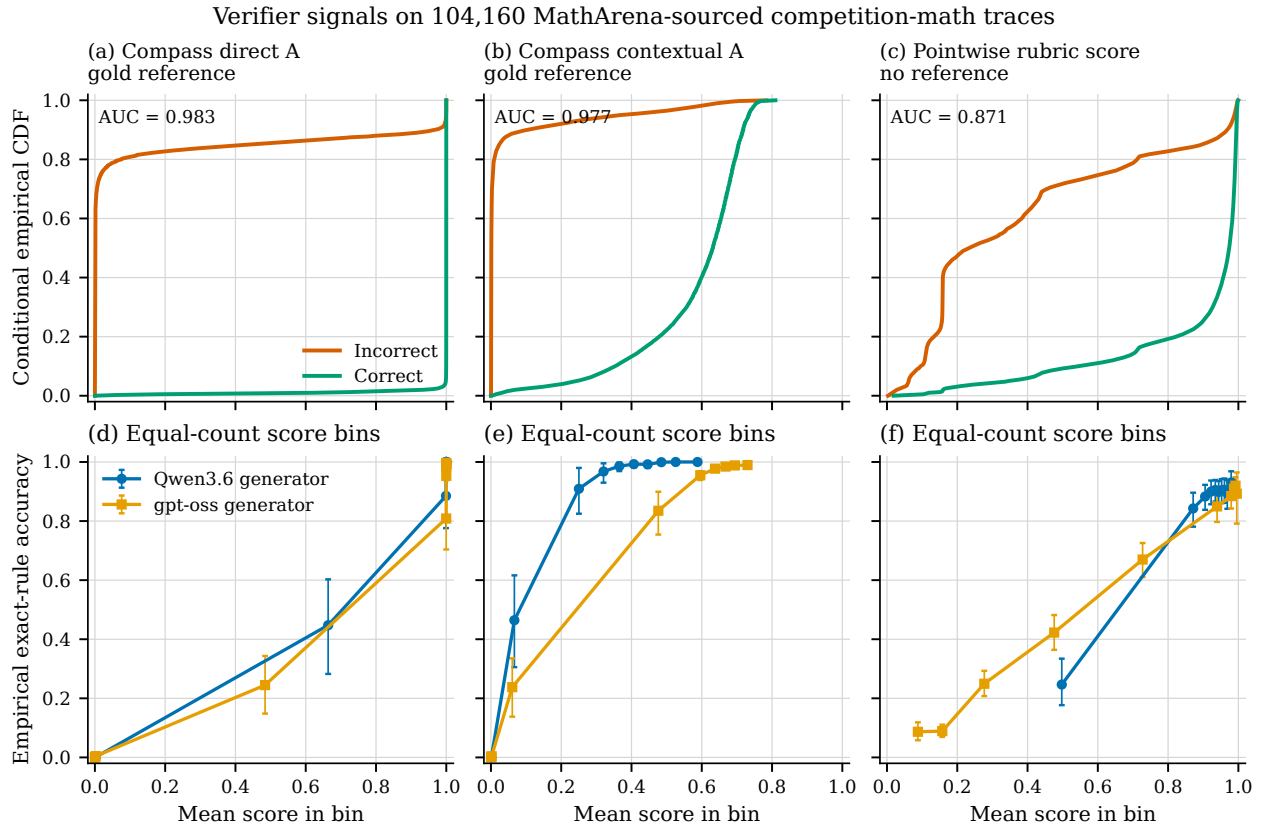


Figure 7: **Verifier signals against the independent rule-based outcome on 104,160 competition-math traces.** The figure compares direct and contextual Compass A scores with the reference-free pointwise Qwen score through conditional distributions and equal-count score bins. The lower panels separate Qwen-generated self-evaluations from gpt-oss cross-model evaluations. Compass sees the reference answer; pointwise Qwen does not. The binned empirical correctness curves describe ranking behavior on this fixed response set and should not be interpreted as calibration guarantees.

Against the rule-based outcome, the descriptive trace-level ROC AUC is 0.983 for direct Compass A, 0.977 for contextual Compass A, and 0.871 for the pointwise score. The pointwise AUC is 0.744 on Qwen-generated traces and 0.876 on gpt-oss traces. This split does not identify a causal self-evaluation effect because the generator families also differ in prompts, output distributions, and base accuracy. A pooled AUC would conceal the observed difference between generators.

Scaling results. Table 10 reports the four canonical banks. String plurality groups literal extracted strings rather than mathematically equivalent forms. Pointwise Best-of- N uses only the reference-free score; Compass scores are excluded from these deployed-selection results.

Canonical bank	Acc.@1	Pass@8	Pass@80	Str.-Plur.@80	Pnt.-BoN@80	Caps
Qwen3.6	83.12	91.70	94.62	89.25	86.56	675
gpt-oss low	32.90	56.14	72.58	45.70	58.06	0
gpt-oss medium	62.80	85.99	91.94	80.11	75.81	32
gpt-oss high	73.03	88.12	93.55	85.48	81.72	2,399

Table 10: Finite-bank and shared-bank results on the 186 MathArena-sourced competition questions, in percent. Caps count responses that reach 81,920 tokens out of 14,880. Pnt.-BoN is pointwise Best-of- N .

At $k = 80$, the 95% prompt-bootstrap intervals for Pass@80 are [90.86, 97.85], [66.13, 79.03], [87.63, 95.70], and [89.78, 96.77] in table order. Pointwise Best-of- N intervals are [81.72, 91.40], [51.08, 65.05], [69.35, 81.72], and [75.81, 87.10]. Several intervals overlap. Mean completion counts per response are 34,382 tokens for Qwen and 1,781, 12,341, and 37,264 for gpt-oss low, medium, and high. These costs exclude prompt tokens and verifier computation. The high-effort cap rate is 16.12%, so the effort comparison is inseparable from the shared length limit.

G.7 Field-balanced SuperGPQA extension

SuperGPQA contains 26,529 multiple-choice questions across 285 graduate disciplines (M-A-P et al., 2025). Our 3,600-question sample includes 50 items from each of 72 EvalScope fields, preventing large fields from dominating the result (ModelScope Team, 2024). The zero-shot prompt requests step-by-step reasoning and a final option letter, which is extracted for binary scoring. The four generation banks are Qwen3.6 and gpt-oss at low, medium, and high effort, with 80 responses per question. Sampling and verification follow the signal-rich mathematics protocol. The pointwise rubric is generalized to factual or technical accuracy, logical coherence, and completeness. Together, the four banks contain $3,600 \times 4 \times 80 = 1,152,000$ responses.

For the gpt-oss high bank, the 288,000 responses have mean accuracy 45.03%. Exact Pass@80 is 81.94%, while pass⁸⁰ is 9.47%. Of the 3,600 questions, 650 are never answered correctly and 341 are answered correctly in all 80 responses. Field-level response accuracy ranges from 21.33% in Aquaculture to 76.78% in Mathematics. Figure 8 reports finite-bank scaling and field variation from the 3,600 by 80 binary matrix. These values are descriptive of the equal-field construction, not estimates under SuperGPQA’s original field frequencies.

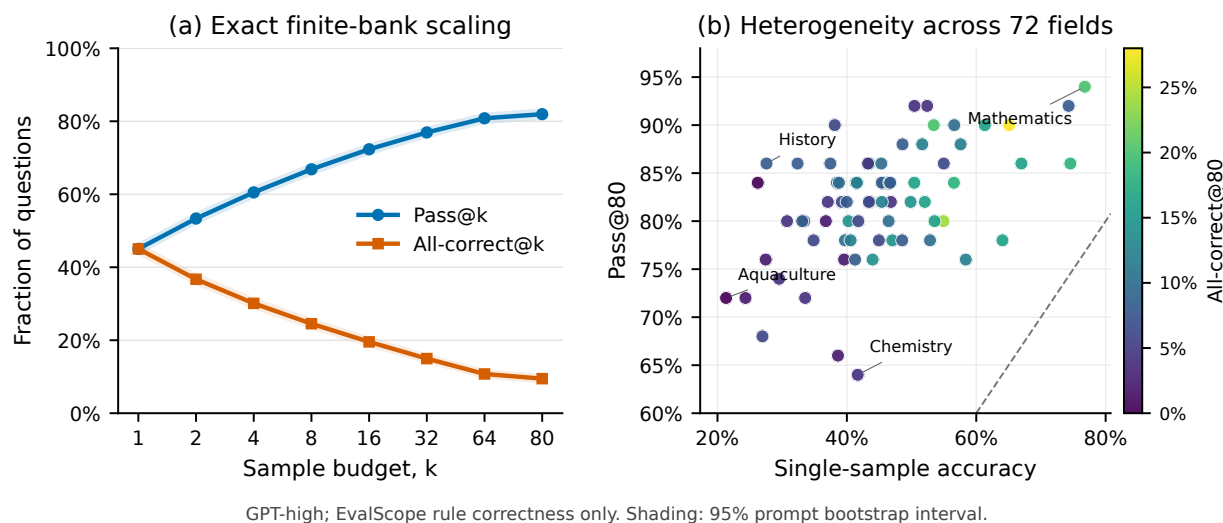


Figure 8: **SuperGPQA scaling and field heterogeneity for the gpt-oss high bank.** The left panel gives exact finite-bank discovery and stability over 3,600 questions. In the right panel, each point is one of 72 equally sized fields: the axes compare single-response accuracy with Pass@80, and color shows the all-correct@80 coordinate. This figure uses only the rule-based correctness matrices; it does not substitute an LLM verifier for the benchmark outcome.